

석사학위 논문

미래전에 대비한
군수분야 UAV 환경을 위한
효율적인 보안 프로토콜 연구

ESPMU: an Efficient Security Protocol
in the Military UAV Environment for the Future War

2020년 02월

순천향대학교 대학원
정보보호학과
고 용 호

미래전에 대비한
군수분야 UAV 환경을 위한
효율적인 보안 프로토콜 연구

ESPMU: an Efficient Security Protocol
in the Military UAV Environment for the Future War

지도교수 유 일 선

이 논문을 석사학위 논문으로 제출함

2019년 12월

순천향대학교 대학원

정보보호학과

고 용 호

고용호의 석사학위논문을 인준함

2019년 12월

위원장 이 선 영 (인)

위 원 유 일 선 (인)

위 원 임 강 빈 (인)

순천향대학교 대학원

목 차

제 1 장 서 론	1
제 1 절 연구의 배경 및 필요성	1
1. 연구의 배경	1
2. 연구의 필요성	4
제 2 절 연구 목적 및 논문의 구성	7
제 2 장 UAV 기존연구 분석	8
제 1 절 UAV 보안 프로토콜 개요	8
1. UAV 보안 프로토콜 개요	8
제 2 절 보안 프로토콜 기존연구 분석	10
1. GEC 프로토콜	10
2. DSD 프로토콜	13
3. UAS 프로토콜	17
4. WDKVR 프로토콜	21
5. 기존연구 비교 분석	25
제 3 장 미래전에 대비한 군수분야 UAV 환경에서의 보안 프로토콜 연구	27
제 1 절 제안 프로토콜 목표 및 가정	27
1. 제안 프로토콜 목표	27
2. 제안 프로토콜 가정	32

제 2 절 제안 프로토콜 설계	36
1. D-GCS 보안 프로토콜	36
2. D-MD 보안 프로토콜	39
 제 4 장 제안 프로토콜 평가	41
제 1 절 Scyther를 이용한 정형화 검증	41
1. Scyther 개요	41
2. 제안 프로토콜 정형화 검증	42
제 2 절 BAN 논리를 이용한 정형화 검증	50
1. BAN 논리 개요	50
2. 제안 프로토콜 정형화 검증	52
제 3 절 제안 프로토콜 비교 및 성능 분석	68
1. 제안 프로토콜 비교	68
2. 제안 프로토콜 성능 분석	70
 제 5 장 결론	72
 참 고 문 헌	74
부 록	79

표 목 차

<표 1-1> 군수분야 UAV 형상에 따른 주요 특성	1
<표 1-2> 군수분야 UAV 중량에 따른 분류	2
<표 1-3> 군수분야 UAV 운용목적에 따른 분류	3
<표 1-4> UAV 보안 위협에 따른 분류	5
<표 2-1> UAV 보안 프로토콜에서 사용되는 표기	8
<표 2-2> GEC 프로토콜 보안 요구사항	10
<표 2-3> DSD 프로토콜 보안 요구사항	13
<표 2-4> WDKVR 프로토콜 보안 요구사항	21
<표 2-5> 기존연구들의 보안 요구사항 비교표	26
<표 3-1> 시스템 구성요소	28
<표 3-2> 패킷 설명	29
<표 3-3> 본 연구의 보안 요구사항	34
<표 3-4> 3장 용어 정리	35
<표 4-1> Claim 이벤트 설명	43
<표 4-2> BAN 논리 표현	50
<표 4-3> BAN 논리 규칙	51
<표 4-4> 기존연구들과 본 프로토콜의 보안 요구사항 비교표 ...	69
<표 4-5> 기존연구들과 본 프로토콜의 연산 오버헤드 비교표 ...	71

그 립 목 차

<그림 2-1> GEC 프로토콜 절차	11
<그림 2-2> 드론 배달 서비스를 지원하는 보안 프레임워크 구성도 ..	14
<그림 2-3> DSD 프로토콜 절차	15
<그림 2-4> UAS 프로토콜 키 스트림 구조	17
<그림 2-5> UAS 프로토콜 키 스트림 갱신	18
<그림 2-6> UAS 프로토콜절차	19
<그림 2-7> 유저 등록 단계	22
<그림 2-8> 인증 및 키 교환 단계	24
<그림 3-1> D-GCS 및 D-MD 시나리오	27
<그림 3-2> D-GCS 보안 프로토콜 패킷 구조	31
<그림 3-3> D-MD 보안 프로토콜 패킷 구조	31
<그림 3-4> DY 위협 모델	34
<그림 3-5> D-GCS 보안 프로토콜	36
<그림 3-6> D-MD 보안 프로토콜	39
<그림 4-1> DH 알고리즘 모델링	43
<그림 4-2> 전역변수 선언 구문	44
<그림 4-3> 도우미(help) 프로토콜 모델링	44
<그림 4-4> D 모델링	45
<그림 4-5> GCS 모델링	46
<그림 4-6> MD 모델링	47
<그림 4-7> Scyther verify result	49
<그림 4-8> D-GCS 프로토콜 Idealization	52
<그림 4-9> D-GCS 프로토콜 Assumption	53
<그림 4-10> D-MD 프로토콜 Idealization	61
<그림 4-11> D-MD 프로토콜 Assumption	61

국 문 요 약

미래전에 대비한 군수분야 UAV 환경을 위한 보안 프로토콜 연구

고 용 호

순천향대학교 일반대학원

정보보호학과

지도교수 유 일 선

미래전은 네트워크 중심 작전환경인 NCOE(Network Centric Operational Environment)하에서 신속한 지휘 및 선제타격이 준비되어야 승리할 수 있는 정보우위의 양상으로 변화되고 있다. 신속한 지휘를 실현하기 위해서는 정보획득 방법을 육상 위주에서 공중 및 해상 등 다양한 조건에서의 수집 방안을 모색해야한다. 이를 실현하기 위한 대표적인 게임 체인저로 UAV(Unmanned Aerial Vehicle)가 주목받고 있다. UAV란, 조종사가 탑승하지 않고 원격 조종 혹은 사전 임무를 입력받아 비행하거나 자율적으로 환경을 인식하고 판단을 통해 비행하는 비행체계를 의미한다. UAV는 민간뿐만 아니라 군수분야에서 정찰용, 전자전용, 공격용 등과 같이 다방면으로 운용이 가능하다. 그러나, UAV가 널리 보급되고 수요가 증가하면서 다양한 취약점을 이용한 공격에 위협받고 있고, 특히, 군수분야에서 UAV는 재밍, 정보유출, 스푸핑 공격에 매우 취약한 것으로 알려져 있다. 이러한 공격과 보안위협에 대응하기 위해 무엇보다 보안 프로토콜에 대한 연구가 중요하지만 관련 연구가 부족한 실정이다. 또한, 군수분야 UAV의 핵심 필수요소로 지목받고 있는 D2D(Drone to Drone) 보안 시나리오에 대한 연구도 시급하다. 이에 본 논문에서는 군수분야 UAV를 위한 보안 프로토콜을 제안한다. 제안 프로토콜 연구를 위해 먼저 군수분야 UAV의 개념을 소개하고, 공격자 모델의 설정을 통해 군수분야 UAV 환경을 위한 보안 요구사항을 정의한다. 그리고, 정의한 보안 요구사항을 바탕으로 각 시나리오에 적합한 보안 프로토콜을 제안한다. 마지막으로, Scyther 및 BAN 논리를 통해 제안 프로토콜의 보안성을 검증하고, 연산 능력을 비교하여 효율성을 검증한다. 제안 프로토콜은 완전 순방향 비밀성이 보장되는 강력한 인증과 키교환, 전자서명 기반의 부인봉쇄 및 책임소재 추적, 안전한 D2D 보안을 통해 UAV 기반의 작전수행과 작전통제를 효과적으로 지원할 수 있을 것으로 기대된다.

주제어: Military UAV, Authentication, Key Management, Security,

Formal Verification, Scyther, BAN Logic

제 1 장 서 론

제 1 절 연구의 배경 및 필요성

1. 연구의 배경

최근 4차 산업혁명의 중심으로 UAV(Unmanned Aerial Vehicle)에 대한 이목이 집중되고 있다[1]. UAV는 유인 항공기에 비해 상대적으로 규모가 작고 비용에 대한 위험 부담률이 적다는 인식을 토대로 세계적으로 여러 나라에서 다양한 민간분야로 이용되고 있다[2]. 또한, 2018 평창 동계올림픽에서 오륜기를 표현하는데 활용되었으며, 농업, 공업과 같은 민간분야 뿐만 아니라 군수분야와 같은 국가의 안보에 직결되는 분야에도 이용되고 있다[3]. 본 논문에서는 군수분야 UAV를 목적으로 연구를 진행하였으며, 본 절에서는 UAV에 대한 배경 및 필요성을 설명한다. 다음 <표 1-1>은 UAV 형상에 따른 주요 특성을 설명한다[4].

<표 1-1> 군수분야 UAV 형상에 따른 주요 특성

구 분	내 용
고정익	• 항공기의 동체에 고정된 날개 특성 • 제한된 지역에서 이착륙 가능 • 날개의 양력을 이용한 비행을 통해 다른 형상에 비해 연료효율이 좋으며, 속도가 더 빠름

회전익	- 회전하는 날개에 의하여 비행에 필요한 양력의 전부 또는 일부를 발생케 하는 비행체 특성 - 다양한 방식의 이·착륙이 가능 - 날개 양력을 사용한 비행으로 타 형상에 비해 연료효율 및 속도가 낮지만, 운용 용이
혼합형	- 고정익 및 회전익의 특성을 보유 - 고속비행 및 수직 이·착륙이 가능 - 날개의 양력을 이용한 배행을 통해 회전익 형상에 비해 연료 효율 우수
멀티 콥터	- 고정익 및 회전익의 특성 보유 - 다양한 방식의 이·착륙 가능 - 여러 개의 날개의 양력을 사용한 비행으로 타 형상에 비해 연료효율이 양호

UAV는 크기에 따라 무게가 25g인 초소형 UAV에서 12,000kg에 40시간 이상을 비행할 수 있는 대형 UAV까지 매우 다양하게 분류된다. UAV는 <표 1-2>와 같이 초소형 UAV, 소형 UAV, 중소형 UAV, 중형 UAV 그리고 대형 UAV으로 구분된다[5].

<표 1-2> 군수분야 UAV 중량에 따른 분류

구 분	내 용
초소형 무인기	15cm 이내 크기로 1인이 손으로 던져서 운용
소형 무인기	1~2명이 휴대하면서 운용

중소형 무인기	차량 1대에 운용자 및 장비를 탑재하여 이동하면서 운용
중형 무인기	단거리급 이상의 무인기
대형 무인기	중거리급 이상의 무인기

결프전 등 전쟁에서 입증된 군수분야 UAV의 전투효과는 오늘날 많은 국가들로 하여금 군수분야 UAV 연구 개발에 박차를 가하도록 하고 있으며 여러 분야에서 UAV를 운용하고자 시도하고 있다. 군수분야 UAV는 운용목적에 따라 정찰/감시, 공격, 전자전, 수송 그리고 통신 중계 등으로 이용된다. <표1-3>은 운용목적에 따른 분류에 대한 설명이다[5-9].

<표 1-3> 군수분야 UAV 운용목적에 따른 분류

구 분	내 용
정찰/감시용	임무수행의 목표 영역에 대한 실시간 감시, 정찰 및 정보 수집과 같은 임무 수행이 가능하며, 수행·행동반경 및 작전운용 가능 시간에 따라 근거리, 단거리, 중거리, 장기체공 UAV로 구분된다.
공격용	UAV로 미사일 및 총기 등과 같은 무기로 무장을 하여 유인 공격기를 대체하여 전투 임무 수행이 가능하다.
전자전용	전자전·통신감청, 전자정보수집, 방향탐지, 기만작전 등의 임무를 수행가능하다.

통신중계용	산악과 같은 접근제한 지역, 장거리 무선통신, 난청으로 인한 제한 지역에서의 공중무선 중계, 긴급 지휘 및 지시 사항 전달 등과 같은 임무수행이 가능하다.
표적용	대공포, 지대공유도탄 등과 같은 사격훈련과 무기체계 연구 개발을 위한 시험평가 등에서 표적용으로 활용 가능하다.
기만용	소모성 무기체계를 통해 적 방공망에 전술적 기만작전을 수행한다, 공격편대군의 임무수행 및 생존성을 증대시키는 역할을 수행한다.
수송용	접근이 어려운 격오지 지역, 적지중심지역, 고립된 지역 등 제한된 위치에 군수품 수송 임무수행 및 부상자 의무후송용의 임무수행이 가능하다.

2. 연구의 필요성

2008년 미국에서 개발한 군사용 UAV인 RQ-140이 러시아 해킹사이트를 통해 기밀영상정보를 유출할 수 있는 악성코드가 26달러에 판매된 이력이 있으며, 2011년 미국의 록히드마틴과 이스라엘이 공동으로 개발한 군사용 UAV인 RQ-170이 이란 영내를 정찰하는 중에 스푸핑 공격을 통해 캡처당하는 사례가 있어 군수분야 UAV에 대한 보안이 중요성이 대두되고 있다[10]. 그리고, 세계 최대 해킹보안 컨퍼런스인 블랙햇 컨퍼런스에서 UAV 해킹에 대한 발표가 진행되어 보안에 대한 중요성이 계속해서 강조되고 있다[11]. 그로인해 UAV 환경에 보안을 지원하기 위해 현재까지 수많은 연구가 진행되었다[12-19]. Galois, Inc에서 개발한 GEC(Galois Embedded Crypto) [12]는 사실상 업계의 표준이라고 알려진 MAVLink에 적용되었고[13-14], ECDH(Elliptic Curve Diffie Hellman)[15] 및 전자서명을 통해 강력한 보안을 지원한다. 또한, Seo

at al.의 WBC(White Box Cryptography)[16]를 이용하여 쇼핑물 구매과정에서의 키를 보호하는 DSD(Delivery Service Drone) 프로토콜[17], Olivier Blazy et al.의 해시체인 키스트림을 이용한 키 업데이트 프로토콜[18], 그리고 Wazid at al.의 암호화기법을 이용하지 않고 XOR 연산 및 해시함수를 이용한 보안 프로토콜 등이 제시되었다[19]. 기존에 제시됐던 UAV에 대한 주요 보안 프로토콜 연구는 주로 민간분야의 환경에 집중되어 있어 군수분야 시나리오에 적합하지 않다. 더욱이, 군수분야 UAV에서 미래전의 게임체인저로 주목받고 있는 군집 드론(Drone Swarm)을 실현하기 위해서 D2D(Drone to Drone)에 대한 보안이 필수적으로 지원이 되어야하지만, 관련 연구가 부족한 실정이다[20]. UAV 주요 보안 위협은 <표1-4>에서와 같이 ‘정보유출’, ‘스푸핑’, ‘재밍’, 캡처 후 키 추출, 악성코드 감염 그리고 불법장비 탑재로 집중되고 있다[21][22].

<표 1-4> UAV 보안 위협에 따른 분류

구 분	내 용
정보유출	UAV가 비행 중에 장착된 센서를 통해 수집된 정보에 대한 정보를 유출하는 공격이 가능하다.
스푸핑	UAV 비행 시 사용하는 신호 조작을 통해 비행 위치 조작과 같은 공격이 가능하다.
재밍	UAV의 센서 수신 대역 내의 주파수에 방해 신호를 송신하는 방식으로 전자 방해 기술의 일종이다.
캡처 후 키 추출	비행 중인 UAV를 캡처한 후 충분한 시간과 장비로 역공학, 메모리분석 그리고 부채널 분석 등과 같은 기법들을 통해 비밀키를 해킹할 수 있다.

악성코드 감염	악성코드 감염으로 인한 제어권한 탈취로 UAV의 대부분의 기능을 오작동 시킬 수 있다. 공격자는 이를 통해 목적지 변경, 공격자에게 수집정보 전달 등과 같은 추가적인 공격이 가능하다.
불법장비 탐재	비인가 UAV의 폭탄 및 생화학 무기 등과 같은 불법장치 탐재를 식별하지 못한다면 사회적 혼란을 초래할 수 있다.

제 2 절 연구 목적 및 논문의 구성

본 논문에서는 군수분야 UAV 환경에서의 안전하고 효율적인 보안 프로토콜에 대한 연구를 위하여 기존의 연구들을 분석한다. 또한, 기존에 진행된 보안 프로토콜에 대한 연구를 기반으로 군수분야 UAV 환경에 적합한 안전하고 효율적인 보안 프로토콜을 제안하고 이를 정형화 검증 도구인 Syther[23]와 BAN 논리[24]를 통해 보안성을 검증하며, 연산 오버헤드를 비교하여 효율성을 검증한다.

본 논문의 구성은 다음과 같다.

제1장에서는 서론으로 연구의 배경 및 필요성을 설명한다. 또한, 연구 목적 및 논문의 구성에 대하여 기술하였다.

제2장에서는 주요 UAV 기반의 보안 프로토콜인 Galois ,Inc, Seo at al, Olivier Blazy et al. 그리고 Wazid at al.의 기법에 대하여 분석한다.

제3장에서는 군수분야 UAV 환경에 적합한 안전하고 효과적인 보안 프로토콜을 제안한다.

제4장에서는 제3장에서 제안한 보안 프로토콜을 정형화 검증 및 성능 평가를 통해 우수성을 확인한다.

제5장에서는 결론 및 향후 연구에 대해 서술한다.

제 2 장 UAV 보안 프로토콜 기존연구 분석

제 1 절 UAV 보안 프로토콜 개요

1. UAV 보안 프로토콜 개요

오늘날 대표적인 무인 이동체 UAV는 사람의 개입이 최소화되고, 자원이 제한된 환경에서 원격으로 제어되거나 미리 정해진 임무를 통해 운용되기 때문에 정보유출의 위험이 있다. 특히 군수분야 UAV는 군수분야라는 특수한 환경으로 국가의 안보와 직결되므로 강력한 보안 요구사항이 필요하다. 본 장에서는 본 연구의 최종 목표인 군수분야 UAV와 관련이 있는 기존연구에 대해 분석하였다. <표 2-1>은 본 장에서 기존연구 설명에 이용될 표기에 대한 정리이다.

<표 2-1> UAV 보안 프로토콜에서 사용되는 표기

표 기	정 리
ID(X)	X의 식별자
PU(X)	X의 공개키
PR(X)	X의 개인키
K(X)	X의 키
S(X)	X의 Salt 값
Sig(X)	X가 생성한 전자서명
G	타원곡선 DH에서의 기준점
SK	비밀키

t	인증 토큰
$e\text{-receipt}$	전자 영수증
WBE_X	X의 화이트 박스 암호화 기법
WBD_X	X의 화이트 박스 복호화 기법
SD_X	감지된 데이터(Sensed Data)의 X번째 블록
$K_{UAV_d}^L$	L번째 키 스트림의 초기키. 초기에는 $K_{UAV_d}^0$ 으로 설정되며, 이후 L의 순서는 랜덤하게 선택됨.
K_i	감지된 데이터의 암호화를 위한 일회용 키. 프로토콜을 처음 시작할 때는 K_0 으로 초기 설정됨. 후속키는 $K_{i+1} = H_{UAV_d}(K_i)$ 을 통해 연산됨.
i_{LastKS}	이전 키 스트림의 마지막 순서
ts	타임 스탬프
MK_X	X의 마스터 키
BIO	생체 정보
σ_X	X의 생체 인증 키
τ_X	X의 공개 재생 파라미터
RTS_X	X의 등록 타임스탬프

제 2 절 UAV 보안 프로토콜 기존 연구 분석

1. GEC 프로토콜

GEC는 SMACCMPIlot(Secure Mathematically-Assured Composition of Control Model Pilot) DARPA(Defense Advanced Research Projects Agency)의 HACMS(High-Assurance Cyber Military Systems) 프로그램에 따라 Rockwell Collins가 주도한 SMACCM 연구 프로젝트의 일부로 개발되었다.

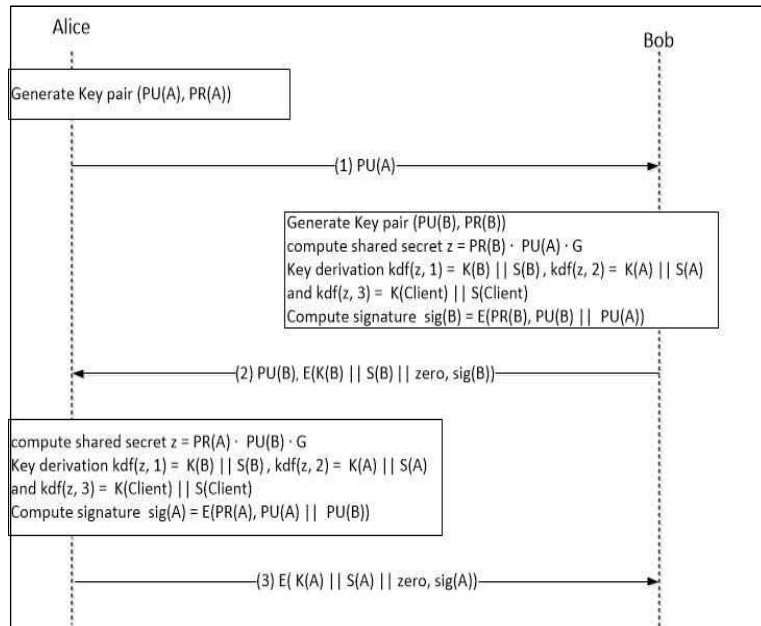
또한, 최근 업계에서 표준이라고 언급되고 있는 MAVLink(Micro Air Vehicle Link)[13] 버전1.0 에서 보안성이 추가된 버전인 SMAVLink(Secure Micro Air Vehicle Link)[14]에서 적용되었으며, 현재 Paparazzi UAV 통신 프로토콜에서 이용되고 있다[25].

GEC 프로토콜에서 설명하는 보안 요구사항으로는 <표2-2>와 같다.

<표 2-2> GEC 프로토콜 보안 요구사항

구 분	내 용
기밀성	고유한 IV 및 비밀키를 통해 AES GCM 알고리즘[26]으로 메시지의 기밀성을 제공한다.
무결성	고유한 IV 및 비밀키를 통해 AES GCM 알고리즘으로 메시지의 무결성을 제공한다.
인증	암호화된 전자서명을 이용하여 상대방을 식별한다.

<그림 2-1>은 GEC 프로토콜의 과정이다. GEC 프로토콜은 기존의 잘 알려진 STS(Station-To-Station) Protocol[27]에서 키 교환을 위한 curve25519[28], 인증을 위한 ed25519[29], 키 유도를 위한 카운터가 있는 SHA-512[30] 및 키 확인을 위한 AES128(카운터 모드)[31]을 결합하여 수행된다.



<그림 2-1> GEC 프로토콜 절차

GEC 프로토콜의 과정은 다음과 같다.

- (1) A는 자신의 ECDH 공개키 및 비밀키를 생성하고 B에게 자신의 공개키를 송신한다.
여기서 첫 번째 메시지의 구성요소는 A의 공개키만 포함되어 송신이 된다.
- (2) B는 첫 번째 메시지를 받게 되면 B의 ECDH 공개키 및 비밀키를 생성한다. 이후, 자신의 개인키와 첫 번째 메시지에서 받은 A의 공개키를 자신의 개인키 및 기준점과 연산하여 고유 비밀 z 를 계산한다. z 를 통해 사전에 서로 공유한 키 유도 함수를 통해 $kdf(z, 1) = K(B) \parallel S(B)$, $kdf(z, 2) = K(A) \parallel S(A)$ 그리고 $kdf(z, 3) = K(Client) \parallel S(Client)$

S(Client)를 계산한다. 다음으로, 개인키를 이용하여 자신의 공개키와 앞서 메시지에서 받은 상대의 공개키를 서명한다. 앞서 설명한 계산이 모두 완료되면 자신의 공개키와 $kdf(z, 1)$ 에서 생성한 $K(B)$, $S(B)$ 그리고 패딩 값으로 채운 값을 이용해 전자서명을 암호화해서 보낸다. 이번 단계에서, B는 첫 번째 메시지를 받게 되면 메시지의 freshness 판단 없이 앞서 설명한 과정들을 진행해야하므로 자원 고갈형 서비스 거부공격에 대한 문제가 있다.

- (3) A는 두 번째 메시지를 받은 이후, 자신의 개인키와 두 번째 메시지에서 받은 B의 공개키를 자신의 개인키 및 기준점과 연산하여 고유 비밀 z 를 계산한다. z 를 통해 사전에 서로 공유한 키 유도 함수를 통해 $kdf(z, 1) = K(B) \parallel S(B)$, $kdf(z, 2) = K(A) \parallel S(A)$ 그리고 $kdf(z, 3) = K(Client) \parallel S(Client)$ 를 계산한다. 여기서 $kdf(z, 1)$ 을 통해 도출된 $K(B)$ 및 $S(B)$ 값을 통해 암호화된 전자서명을 검증함으로써 상대를 인증한다. 다음으로, 개인키를 이용하여 자신의 공개키와 앞서 메시지에서 받은 상대의 공개키를 서명한다. 앞서 설명한 계산이 모두 완료되면 자신의 공개키와 $kdf(z, 2)$ 에서 생성한 $K(A)$, $S(A)$ 그리고 패딩 값으로 채운 값을 이용해 전자서명을 암호화해서 보낸다.
- (4) B가 세 번째 메시지를 받게 되면 공유 비밀이 정상적으로 됐는지 확인을 통해 상대를 인증하고 $kdf(z, 3)$ 을 통해 도출된 $K(Client)$ 및 $S(Client)$ 을 이용하여 대칭키를 설정한다.

2. DSD 프로토콜

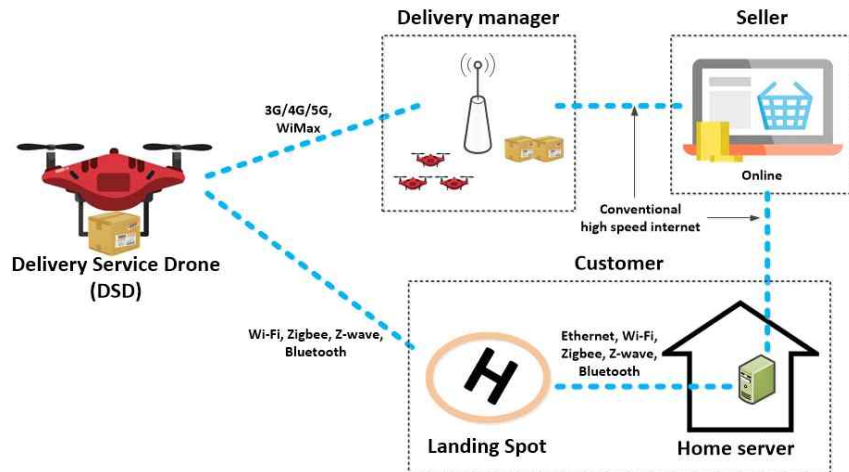
<그림 2-2>는 Seo et al.이 제안한 드론 배달 서비스를 위한 보안 프레임워크를 나타낸다. Seo et al.의 기법은 인증 및 부인 방지 방법으로 PKI(Public Key Infrastructure)와 함께 무인 항공기 및 착륙 지점의 암호화 자산에 대한 소프트웨어 보호 도구로 WBC 구현을 활용하는 것에 중점을 두었다.

드론 배달 서비스를 지원하는 보안 프레임워크에서 DSD 프로토콜의 보안 요구사항은 <표 2-3>과 같다.

<표 2-3> DSD 프로토콜 보안 요구사항

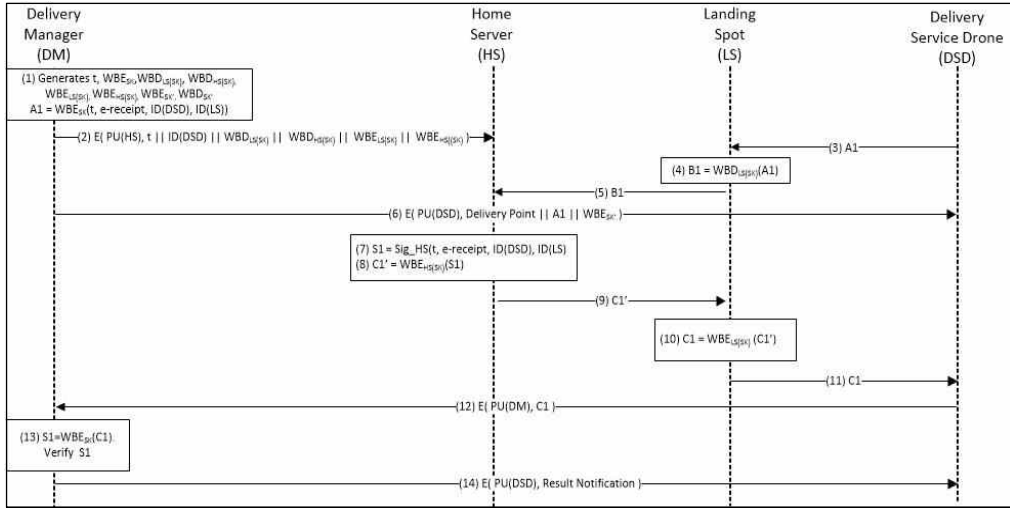
구 분	내 용
기밀성	고유한 IV 및 비밀키를 통해 AES GCM 알고리즘으로 메시지의 기밀성을 제공한다.
무결성	고유한 IV 및 비밀키를 통해 AES GCM 알고리즘으로 메시지의 무결성을 제공한다.
인증	암호화된 전자서명을 이용하여 상대방을 식별한다.
부인방지	ECDSA(Elliptic Curve Digital Signature Algorithm)[32] 및 RSA(Rivest Shamir Adleman)와 같은 디지털 서명 구성표가 사용된다.
소프트웨어 보호	WBC를 통해 무인 항공기 및 착륙 지점에 저장된 중요 데이터를 보호한다.

<그림 2-2>는 DSD 프로토콜의 시스템에 대한 구성도이다. 온라인 쇼핑물을 통한 구매과정 시나리오를 통해 설계되었으며, 각 피어에 접근하기 위한 무선 네트워크에 대해 도식화하였다.



<그림 2-2> 드론 배달 서비스를 지원하는 보안 프레임워크 구성도

<그림 2-3>에서처럼 DSD 프로토콜은 크게 사전 설정, 배달, 검증으로 3 단계로 구성된다.



<그림 2-3> DSD 프로토콜 절차

- (1) 첫 번째 단계는 사전 설정 단계이다. 사전 설정 단계에서 구매자는 온라인 쇼핑몰에서 상품을 구매하고 ECDSA를 사용하여 전자 영수증에 디지털 서명을 생성한다. 이후, 판매자는 구매자의 주소 및 전자 영수증과 같은 배달 정보를 배달 관리자에게 전달하고 배달 관리자는 배달 서비스에 대한 인증 토큰 t , 고객에 대한 비밀키 SK' 를 포함하는 WBE 모듈 $WBE_{SK'}$, WBE 및 WBD 모듈을 포함한다.
- (2) 두 번째 단계는 배달 단계이다. DSD는 전송 임무 중에 배달 상태 및 비행 데이터에 대한 로그를 저장해야하므로 무인 항공기가 캡처 될 때 데이터가 공격자에게 유출되지 않도록 데이터를 $WBE_{SK'}$ 을 통해 암호화를 진행한다. 사전에 공유된 토큰 t 을 이용하여 SSL과 같은 보안 채널을 통해 값들을 안전하게 송신한다. 배달 단계에서는 구매자의 개인 주소와 인증 토큰 같은 민감한 정보들이 포함되기 때문에 캡처공격으로 부터 보호되어야한다. 그러므로 배달 관리자는 $A1$ 으로 표기된 $WBE_{SK'}$ 를 이용하여 토큰 t , 전자 영수증, $ID(DSD)$ 및 $ID(LS)$ 을 암호화하고 기지국 내의 배달 지점 LS , $A1$ 및 $WBE_{SK'}$ 을 DSD 에 삽입한다. LS 는 수신 된 메시지 $A1$ 에 $WBD_{LS}(SK)$ 를 적용하고

부분적으로 해독된 데이터 B1을 생성한다. LS는 부분적으로 해독된 데이터 B1을 HS로 전송하게됨. 이후 HS는 WBDHS(SK)를 사용하여 B1에서 토큰 t , 전자 영수증, ID(DSD) 및 ID(LS)를 복구하고 복구된 토큰 t 및 ID(DSD)를 구매 거래 단계에서 수신되어 저장된 토큰 t 및 ID(DSD)와 비교한다. 구매자는 토큰 t 을 통해 DSD를 인증한다. 구매자의 HS는 ECDSA를 고객의 개인키로 사용하여 전자 영수증을 포함한 복구된 데이터에 대한 서명데이터 S1을 생성한다. 구매자는 WBEHS(SK), $C1' = WBEHS(SK)(S1)$ 을 사용하여 S1을 암호화함. 이후 HS는 $C1'$ 을 LS에게 보낸다. LS가 메시지를 수신하면 WBELS(SK)을 이용하여 암호화된 데이터 C1을 생성하고 DSD에게 보낸다. 여기서 잘못된 착륙 지점에서 배달 물건을 내리는 경우 UAV는 정확한 C1을 받을 수 없다. 마지막으로, DSD가 임계 시간 안에 C1을 수신하지 못하면 DSD는 관리자에게 연락을 하거나 사진을 찍는 것과 같은 후속 조치를 수행한다.

- (3) 마지막 3단계는 검증 단계이다. 검증 단계에서 DSD는 암호화된 데이터 C1을 배송 관리자에게 전송하고 응답을 기다린다. 배송 관리자는 WBDSK를 이용하여 C1에서 S1을 복구하고 고객의 공개키를 사용하여 서명 데이터 S1을 확인하고 토큰 t , 전자 영수증, ID(DSD) 및 ID(LS)를 확인한다. 확인 후 배송 관리자는 서명 확인 결과를 DSD으로 전송함. 결과가 성공이면 UAV는 기지국으로 돌아가고 서명 S1으로 구매자에 배송 물건을 받았음에 대한 부인방지를 한다. 만약, 결과가 실패이면 DSD는 관리자를 호출하거나 사진을 찍는 등 후속 조치를 수행한다. 확인 후 배송 관리자는 최종적으로 검증 결과를 판매자에게 송신한다.

3. UAS 프로토콜

Olivier Blazy et al.은 다음 보안 요구사항을 고려하여 UAV를 위한 보안 프로토콜을 제안하였다(이하 UAS 프로토콜).

- 각 UAV에는 자체 암호화 수단인 키가 있어야한다.
- UAV의 캡처 및 포렌식은 UAS의 보안을 손상시키지 않아야한다.
- PFS(Perfect Forward Secrecy) 및 PBS(Perfect Backward Secrecy)를 보장하기 위해 임무 수행 중에 열쇠가 파생되어야한다.
- C2(Command and Control) 링크를 사용하여 암호화 방법을 수시로 갱신해야한다.
- 잠재적인 손실을 피하기 위해 연결이 가능하자마자 수집된 데이터를 GCS(Ground Control Station)에 보내야한다.
- GCS는 안전하다고 가정한다.

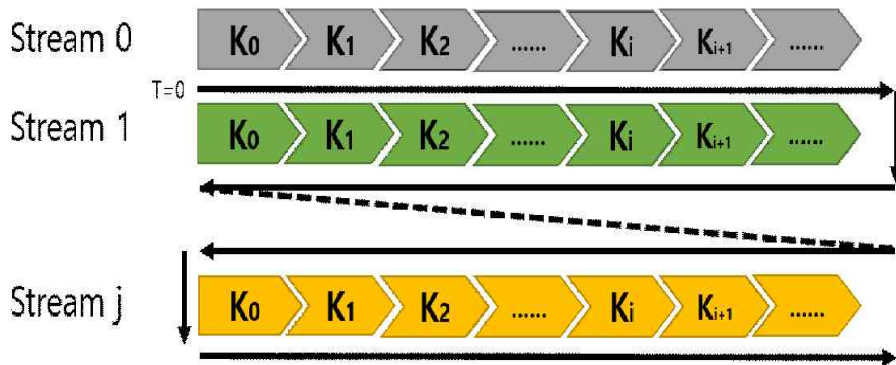
Olivier Blazy et al. 기법의 보안 요구사항에서 언급된 PFS와 PBS를 보장하기 위해 <그림 2-4>와 같이 키 스트림을 이용한다.



<그림 2-4> UAS 프로토콜 키 스트림 구조

첫 번째 키인 K_0 를 기반으로 하여 함수를 통해 후속키를 파생하는 키 스트림이다. $K_{i+1} = H_{UAV}(K_i)$ 와 같이 UAV의 ID를 이용하여 다양화된 해시 함수를 이용한다.

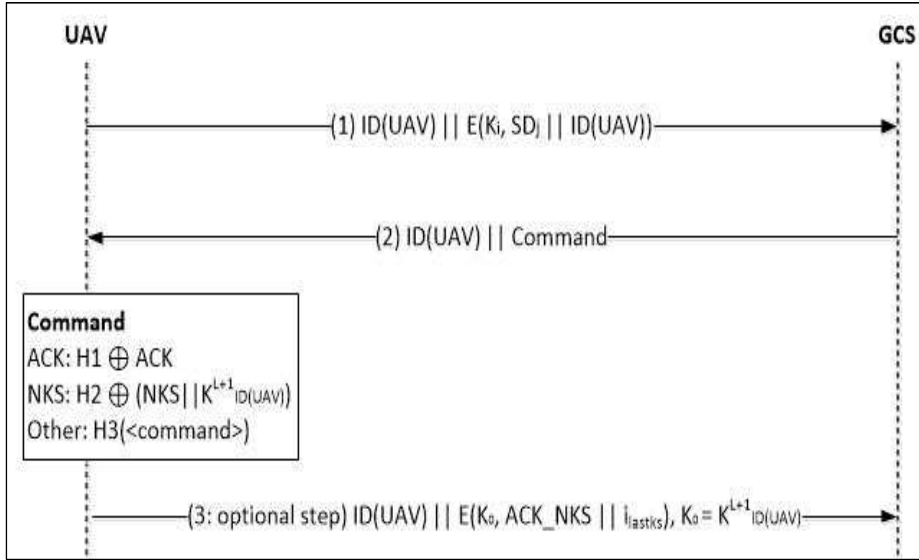
<그림 2-5>와 같이 공격자가 하나의 키를 알고 있는 경우 스트림에서 후속키를 추적할 수 있기 때문에, 키 스트림은 주기적으로 업데이트된다.



<그림 2-5> UAS 프로토콜 키 스트림 갱신

UAS 프로토콜에서 각 키는 데이터 암호화에 한 번만 이용되는 일회성 키다. 각 키는 $H_1 = H(K_i || 1)$, $H_2 = H(K_i || 2)$, $H_3 = H(K_i || 3)$ 와 같이 연산되어 인증 티켓 트리플렛 (H_1, H_2, H_3) 으로 사용된다. 인증티켓은 이후 C2 프로토콜에서 이용된다. 또한, 스트림의 후속키를 생성하고, 메모리에서 삭제하여 복구할 수 없도록 한다.

다음의 <그림 2-6>은 UAS 프로토콜 절차다. 여기서, 각 UAV는 첫 번째 키 스트림의 원점으로 사전 구성된다. : $K_0 = K_{UAV_D}^0$ 또한, GS는 UAS의 각 UAV에 대한 첫 번째 키 스트림으로 사전 구성된다.



<그림 2-6> UAS 프로토콜 절차

UAV가 임무수행을 하는 동안 센싱 및 암호화 프로세스는 다음과 같이 진행된다. 각각의 감지 된 데이터 블록(SD_j)은 즉시 암호화되어 현재의 키(K_i)를 사용하여 UAV의 비휘발성 메모리에 저장된다. SD_j 는 K_i 를 사용하는 효율적인 대칭 알고리즘으로 암호화되며 그 결과 $[SD_j || ID(UAV)]^{K_i}$ 는 비휘발성 메모리에 저장된다. $ID(UAV)$ 가 암호화된 데이터에 추가되어 UAV에서 올 때 GCS가 결과에 의미가 있는지 확인할 수 있다. 위의 암호화에 대해 UAV는 인증티켓 트리플렛 (H_1, H_2, H_3)를 연산하여 저장해야한다. 여기서, 이 티켓은 나중에 C2 링크의 복호화 command에 사용된다. 후속키 K_{i+1} 이 연산되고 현재 사용키 K_i 가 메모리에서 삭제된다.

UAV가 임무수행을 하는 동안 통신 프로세스는 다음과 같다. UAV가 GS의 통신 범위에 있으면 사용 가능한 암호화된 데이터를 전송한다. $[SD_j || ID(UAV)]^{K_i} \dots, [SD_{j+n} || ID(UAV)]^{K_{i+n}}$ 그리고 GCS로부터 인증 된 명령을 수신 할 때까지 이들을

유지한다. 암호화 된 SD는 하나의 인증 된 명령을 필요로 한다. UAV가 관련 인증 명령을 받지 못하면 UAV는 암호화 된 데이터를 수신 할 때까지 반복해서 전송한다. UAV는 GCS로부터 명령을 받으면 연산 된 인증티켓 트리플렛 (H_1, H_2, H_3)으로 인증한다. 그러면 명령 인증에 사용 된 티켓과 관련된 세 겹으로 승인된 암호화된 데이터가 메모리에서 삭제된다. Olivier Blazy et al.의 기법에는 ACK, NKS(New Key Stream) 그리고 C2 세 가지 유형의 명령이 있다. ACK 명령은 GCS에서 데이터 수신을 확인하는 데만 사용된다. NKS(New Key Stream) 명령은 키 스트림을 새로운 것으로 변경함. 새로운 원점이 명령과 함께 제공된다. 여기서 UAV가 확인해야하는 이 특정 명령에 대해 일부 비동기 공격을 방지하기 위한 참고 사항은 키 스트림 변경이 있음을 나타낸다. 마지막 옵션은 일반적인 C2 명령이다.

4. WDKVR 프로토콜

Wazid et al.은 IoD(Internet of Drone) 환경을 위한 경량의 안전한 원격 사용자 인증 및 키교환 프로토콜을 제안하였다(이하 WDKVR 프로토콜).

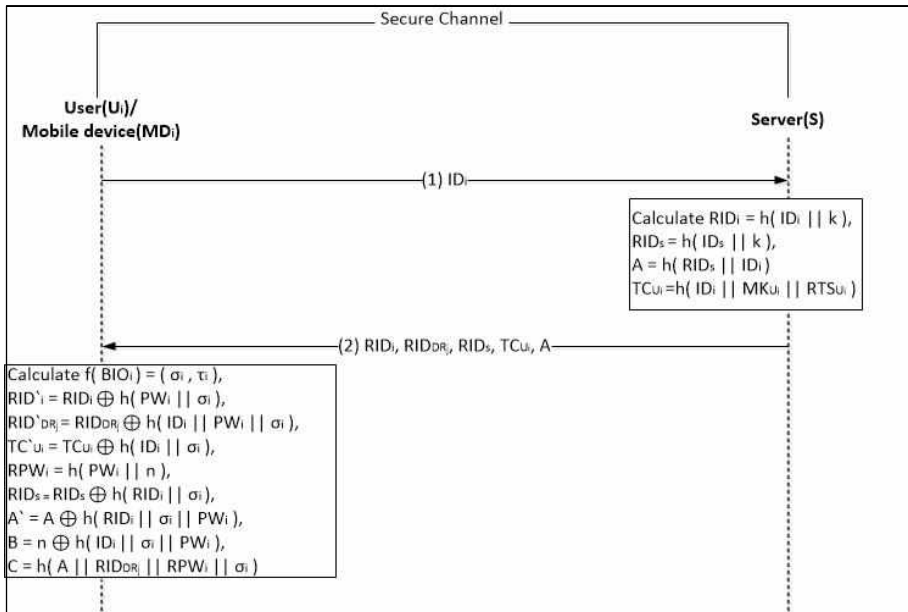
WDKVR 프로토콜은 IoD 환경에서 실시간 데이터 액세스를 가정하여 제안되었다. 또한, 모든 네트워크가 시간동기화가 되어있다고 가정하였고, 랜덤 넘스 값과 타임스탬프를 이용하여 재사용 공격을 방지하며, 해시함수, XOR연산 그리고 생체인식을 이용하여 보안성을 지원하였다.

WDKVR 프로토콜의 기법에서 설명된 보안 요구사항은 <표 2-4>와 같다.

<표 2-4> WDKVR 프로토콜 보안 요구사항

구 분	내 용
기밀성	메시지를 일방향 해시를 통해 연산하여 전송하기 때문에 공격자는 내용을 알 수 없다.
무결성	타임스탬프를 이용하기 때문에 타임 윈도우 외에는 공격하기가 어려움이 있다.
상호인증	사전에 안전이 보장되는 연결 상태에서 연산하는 방법을 나누었기 때문에 연산된 값에 대한 비교를 통해 유저와 UAV 간 상호인증이 가능하다.

초기 설정 단계에서 서버 S에서는 시작 전에 각 무인장치 DR_j 를 등록해야한다. S는 고유한 160 비트 시크릿 넘버 k 및 DR_j 에 대한 고유 식별 ID_{DR_j} 를 먼저 선택한다. 이후, Pseudo 식별자를 $RID_{DR_j} = h(ID_{DR_j} || k)$ 와 같이 연산한다. S는 DR_j 에 대응하는 160비트의 마스터키 MK_{DR_j} 를 선택한다. 또한, 시간에 대한 자격 증명을 $TC_{DR_j} = h(ID_{DR_j} || MK_{DR_j} || RTS_{DR_j})$ 로 연산한다. 여기서 RTS_{DR_j} 은 DR_j 의 등록된 타임 스탬프다. 다음으로 인접한 드론은 양방향 키 설정을 진행한다. S가 RID_{DR_j} 에 대응하는 임시 식별자 TID_{DR_j} 를 생성한다. 이후, Galois field GF(p)를 이용하여 다항식 $P(D_{DR_j}, y)$ 를 연산한 뒤, 연산된 모든 값들을 저장하여 종료한다.

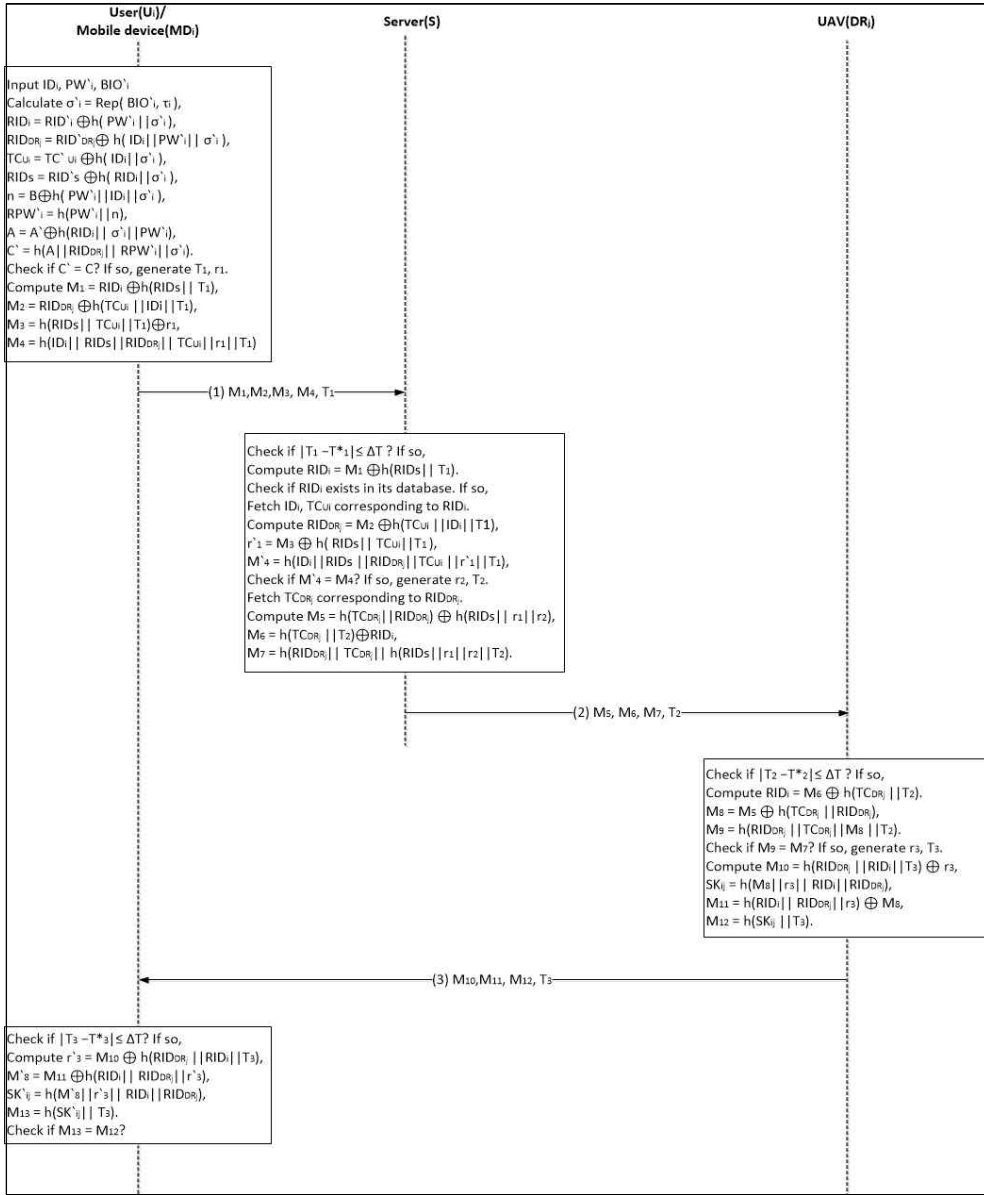


<그림 2-7> 유저 등록 단계

<그림 2-7>의 유저 등록 단계는 외부 사용자에게 대한 등록 절차이다. 유저는 자신의 신원 ID_i 를 등록 요청 메시지를 S에게 보호된 채널을 통해 전송한다. 이후,

S는 160 비트의 시크릿 넘버 k 를 이용하여 유저 신원을 RID_i 으로 연산하고, 자신의 신원을 RID_s 그리고 A로 연산한다. 또한 U_i 의 임시 자격 증명인 TC_{U_i} 을 발급한다. 여기서, MK_{U_i} 는 U_i 및 RTS_{DR_j} 에서 k 를 추가하여 생성된 타임스탬프이다. S로부터 등록 응답을 수신하면 U_i 는 자신의 패스워드 및 생체정보를 입력하여 퍼지 추출방법을 통해 한 쌍의 값을 구한다. 이후, <그림 2-7>의 마지막 단계와 같이 연산한 뒤, 결과 값들을 저장한다.

<그림 2-8>은 인증 및 키교환 단계에 대한 프로토콜이다. 처음 로그인을 시도할 때 생체 인식 값 BIO를 모바일 디바이스의 인터페이스에 입력하고 오류 허용 값에 들어간다면 생체 인식키를 생성한다. 다음으로 $C'=C$ 인지를 확인하여 U_i 는 패스워드 및 생체 인증을 통과한다. 이후, 타임스탬프와 넌스 값을 생성하고 해시함수를 통해 갖고 있는 값들을 연산하여 전송한다. S는 메시지를 수신하면 타임스탬프를 확인하여 메시지가 fresh한지 검증함. 여부가 끝나면 유저와 UAV 간 안전한 통신을 지원하기 위해 세션키를 확립한다. 조건이 유효하면 갖고 있는 값들을 통해 RID_i 값에 대응하는 ID_i 및 TC_{U_i} 를 도출해내고 이를 통해 $M'_4 = M_4$ 를 검증하여 S는 유저를 인증한다. 이후, S는 타임스탬프와 넌스 값을 생성하고 연산된 값들과 함께 UAV에게 전송한다. UAV는 메시지를 수신하면 타임스탬프를 확인하여 메시지가 fresh한지 검증한다. 여기서, 적시성이 확인되면 UAV는 $M_9 = M_7$ 검증을 통해 S를 검증한다. 이후, 타임스탬프와 넌스를 생성하고 연산된 값들과 함께 모바일 디바이스에 전송한다. 마지막으로 모바일 디바이스에서는 수신된 메시지의 타임스탬프를 확인하여 메시지가 fresh한지 검증한다. 여기서 U_i 는 세션키 SK'_{ij} 를 연산하고 $M_{13} = M_{12}$ 를 검증하여 일치하는 경우 UAV를 인증되고 SK'_{ij} 가 제대로 공유되었다고 판단하여 사용한다.



<그림 2-8> 인증 및 키교환 단계

키 분배가 완료된 이후 드론끼리 D2D(Drone to Drone) 통신이 하고 싶을 땐 서로 TID 값을 공유하여 $SK_{DR_j, DR_k} = P(TID_{DR_j}, TID_{DR_k})$ 와 같이 사전에 공유된 다항식 함수를 통해 세션키를 공유한다.

5. 기존연구 비교 분석

다음의 <표 2-5>는 본 장에서 분석한 기존연구들과의 보안성에 대한 비교표이다. Galios, inc의 GEC는 D-GCS(Drone to GCS)의 시나리오위주의 연구가 진행되었으며, D2D 시나리오의 언급은 없다. 또한, PFS 및 PBS를 고려하였지만 과도한 연산이 필요하기 때문에 비행 중 키 업데이트에 큰 부담을 준다. 또한, 자원 고갈형 서비스 거부 공격에 대한 면역을 고려하지 않아 긴박한 상황에서의 임무수행을 고려해야하는 군수분야 시나리오에 적합하지 않다.

Seo et al.의 DSD 프로토콜은 온라인 쇼핑물의 구매과정 시나리오를 통해 연구가 진행되었으며 D2D에 대한 언급은 없다. 화이트박스를 이용하는 방안은 키를 보호하는 데 강력한 강점을 갖고 있다. 하지만, 연산 오버헤드가 과도하기 때문에 본 연구와는 맞지 않으며 암호화 모듈 배포문제가 있다.

Olivier Blazy et al.의 UAS 프로토콜은 D-GCS 시나리오에서 해시체인 원리의 키 스트림을 이용한 키갱신을 통해 PFS 및 PBS를 충족하였지만, 초기인증 및 상호인증에 대한 언급이 없으며, 군수분야 시나리오에서는 부인봉쇄에 대한 요구 사항이 필수적으로 필요하기 때문에 적절하지 않다.

마지막으로, Wazid et al.의 WDKVR 프로토콜은 XOR 및 해시만을 이용하는 방안을 통해 경량화된 기법을 제안하였으나, 연산과정이 복잡하고 반복적이며 사용자의 바이오 인증을 해야 하는 문제점을 갖는다. 또한, 부인봉쇄 및 PFS, PBS를 지원하지 않으므로 군수분야 시나리오에 적절하지 않다. 그리고, D2D를 언급하였으나 기존의 함수를 공유하여 각 참여자의 ID값의 입력을 통해 키를 계산하는 방식을 이용하므로 키를 올바르게 계산할 수는 있지만, 올바른 사용자인지 판단할 수 없다.

<표 2-5> 기존연구들의 보안 요구사항 비교표

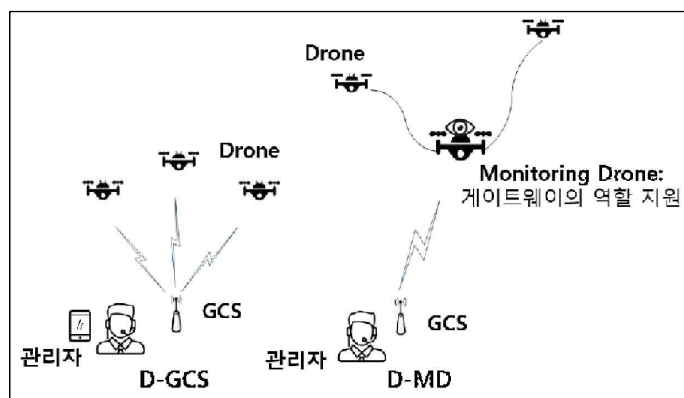
	GEC 프로토콜 [12]	DSD 프로토콜 [17]	UAS 프로토콜 [18]	WDKVR 프로토콜 [19]
기밀성	○	○	○	○
무결성	○	○	○	○
상호인증	○	○	○	○
부인봉쇄	○	○	X	X
PFS	○	X	○	X
PBS	○	X	○	X
서비스거부 공격 대응	X	○	X	○
중간자 공격 대응	○	○	○	○
D2D 보안 지원	X	X	X	△
Note: ○: 지원함 X: 지원하지 않음 △: 지원하지만, 미흡함				

제 3 장 군수분야 UAV 환경에서의 보안 프로토콜 제안

제 1 절 제안 프로토콜 목표 및 가정

1. 제안 프로토콜 목표

앞서 설명한바와 같이 군집 드론이 미래전의 게임체인저가 될 것으로 전망되고 있어 일반적으로 UAV와 GCS가 통신하는 방식 외에 UAV끼리 통신을 하는 D2D 통신에 대한 연구의 필요성이 강조되었다. 본 논문에서 제안하는 보안 프로토콜은 군수분야 시나리오를 목표로 설계되었으며 <그림 3-1>과 같이 일반적인 방식의 보안 프로토콜인 D-GCS 및 자원이 UAV간 D2D 통신에 보안을 지원하는 D-MD와 같은 두 가지의 시나리오로 설계하였다. 다음의 <표 3-1>은 본 연구의 프로토콜에 시스템의 구성요소다.



<그림 3-1> D-GCS 및 D-MD 시나리오

<표 3-1> 시스템 구성요소

구성요소	설 명
 : 관리자	기지국 GCS을 통해 Drone 혹은 모니터링 Drone에 명령을 입력하는 담당자
 : GCS	관리자를 통해 명령을 입력받는 제어 시스템
 : Drone	GCS를 통해 제어되는 일반 Drone 혹은 성능이 낮은 일반 Drone을 뜻함
 : 모니터링 Drone	D2D 통신에서 성능이 낮은 일반 Drone을 대신 데이터를 수집해 GCS에게 송신하는 Drone으로 게이트웨이의 역할을 지원한다
CMD	관리자에 의해 입력되는 명령
ENC(D)	D2D 통신을 지원하기 위해 옵션으로 발행되는 세션키의 암호화된 형태
ST(D)	D2D 통신을 지원하기 위해 옵션으로 발행되는 인증 티켓

시나리오를 시작하기 전에 관리자가 출격할 UAV를 격납고에서 관리하고 있는 관리자에게 직접 전달하여 출격한 UAV를 선택하고 명령을 GCS에 입력한다. 선택된 UAV는 GCS에게 인증을 시도하게 되고 이후, GCS와 상호인증을 진행하고 키를 파생한 뒤, 키를 통해 임무를 하달 받게 된다. 추후에 D2D 통신이 필요하다면 옵션으로 이에 대한 티켓 및 세션키도 함께 발행한다. D-MD는 앞서 설명한 시나리오를 통해 티켓과 세션키를 얻은 일반 UAV가 모니터링 UAV에 인증요청을 하게 되고 모니터링 UAV는 자신에 대한 인증을 요청한 뒤 키 파생 이후 검증을

통해 보안통신을 연결하게 된다.

D-GCS 환경에서는 UAV와 GCS가 직접 통신을 통해 UAV의 상태를 HeartBeat 메시지를 통해 보고 받으며, 명령을 직접 전달받을 수 있다. D-MD 환경에서는 능력이 제한되는 일반 UAV와 GCS 사이에서 모니터링 UAV가 일반 UAV들의 상태를 집계하여 GCS에 전달해주는 게이트웨이의 역할을 수행한다.

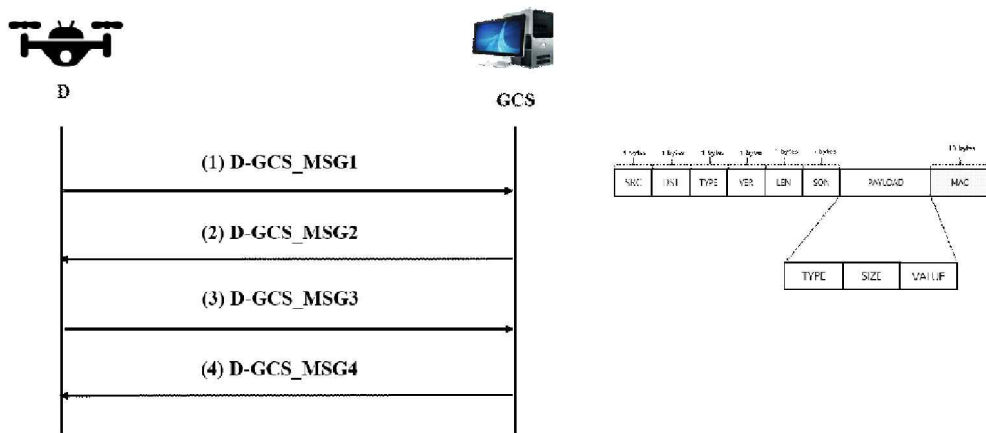
<그림 3-2>와 <그림 3-3>은 앞서 설명한 두 가지의 시나리오에 대한 패킷 구조다. D-GCS 프로토콜은 4단계의 메시지로 구성되며, D-MD 메시지를 3단계로 설계하였다. 또한, 본 구조는 업계에서 사실상 표준이라고 인정받고 있는 MAVLink와 동일한 포맷을 이용하여 호환이 가능하도록 설계되었다. <표 3-2>는 패킷에 이용되는 구성요소의 설명이다.

<표 3-2> 패킷 설명

Byte Index	구성요소	설 명
0	SRC	패킷을 시작점을 알리는 패킷
1	DST	패킷을 도착점을 알리는 패킷

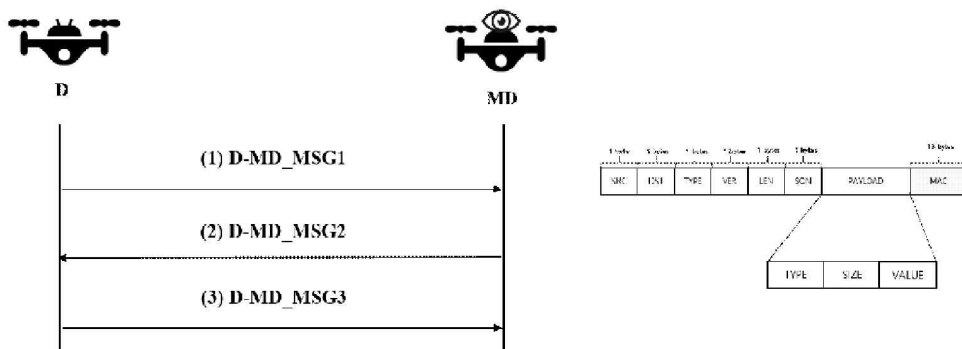
2	TYPE	패킷의 유형을 알리는 패킷
3	VER	패킷의 버전을 알리는 패킷
4	LEN	패킷의 길이를 나타내는 패킷
5	SEQN	패킷의 시퀀스 넘버를 나타내는 패킷
6 to (n+6)	PAYLOAD	메시지 데이터로 메시지의 유형(예: 메시지ID), 페이로드의 길이, 내용을 나타내는 패킷
(n+7) to (n+8)	MAC	메시지 인증 코드

D-GCS 시나리오는 앞서 설명한바와 같이 4단계의 메시지로 구성된다. 첫 번째 메시지에서 UAV는 GCS에게 자신을 요청하며 인증을 요청하게 되고 2번에서 GCS가 요청에 응하면서 상호인증이 진행된다. 또한, 2, 3단계에서 키 파생을 하여 3단계에서 검증을 통해 강력한 키교환을 지원한다. 마지막으로 2, 3, 4단계를 통해 임무를 하달하고 UAV가 이것에 대해 컨펌을 하여 임무 CMD에 대한 인증을 진행한다. 만약, UAV가 D2D 통신을 할 예정이라면 3, 4단계에서 옵션으로 모니터링 UAV에 인증할 수 있는 인증티켓 ST와 세션키 SK를 추가로 발급한다.



<그림 3-2> D-GCS 보안 프로토콜 패킷 구조

D-MD 메시지는 앞서 설명한바와 같이 3단계로 구성된다. 첫 번째 단계에서 일반 UAV는 D-GCS 프로토콜에서 인증 티켓 ST와 세션키 SK를 통해 본인을 인증한다. 이후 두 번째 단계에서 모니터링 UAV은 일반 UAV가 SK를 갖고 있음을 통해 정상적인 UAV임을 확인하고 인증을 요청하여 상호인증을 진행한다. 또한, 2, 3단계에서 키 파생을 진행하고 보안 채널형성을 통해 보안을 지원한다.



<그림 3-3> D-MD 보안 프로토콜 패킷 구조

2. 제안 프로토콜 가정

제안된 보안 프로토콜의 가정은 다음과 같다.

- GCS와 모든 소속 UAV는 현재 시간을 나타내는 타임스탬프 값을 구할 수 있고, 시간동기화가 되어 있어 상대방으로부터 주어진 타임스탬프 값을 검증할 수 있다.
- GCS와 모든 소속 UAV는 타원곡선 전자서명 알고리즘 ECDSA 공개키를 지원하는 공개키 쌍과 인증서를 소유하고 있다.
- 사용자는 작전 수행 전에 관련 응용프로그램을 통해 작전을 계획하고 작전에 포함된 UAV를 선택함. 이때, 작전 아이디 IDMission과 PIN 번호를 부여받는다.

컴퓨팅에서 위협은 시스템에 손실이나 피해를 줄 수 있는 모든 유형들로 이해 될 수 있다. 기본적으로 위협이란, 시스템의 기밀성, 무결성 및 가용성과 같은 요구사항들에 대한 위반을 대상으로 하는 이벤트를 뜻한다. 이러한 위협은 실제로 설계 결함, 구성의 실책, 보안 정책 부정확성 등으로 인한 시스템의 약점인 여러 가지 취약점으로 인해 발생할 수 있다. 따라서 악의적인 의도와 기술적 능력을 가진 사람은 누구나 이러한 취약점을 악용하여 공격을 시작할 수 있다. 이러한 공격은 내부자 또는 외부자일 수 있는데, 전자는 합법적이거나 허가된 시스템에 액세스 권한을 가진 적에 의해 수행되는 재생, 위조 및 변조, 부인, 장애 등과 같은 악의적인 공격을 말한다.

후자는 시스템 액세스 권한이 있는 사람이 시스템 네트워크나 컴퓨터 시스템에 가한 공격을 나타낸다. 외부의 위협은 방화벽 외부에 있는 공격 영역의 보안 노출을 악용하려는 악의적인 캠페인 및 위협 행위자다. 인터넷에 있는 모든 조직은 인터넷, 소셜 미디어 및 모바일 앱을 통해 브랜드 및 공식 커뮤니케이션 채널을 사칭하려고하는 공격자에 의해 외부 위협에 노출된다.

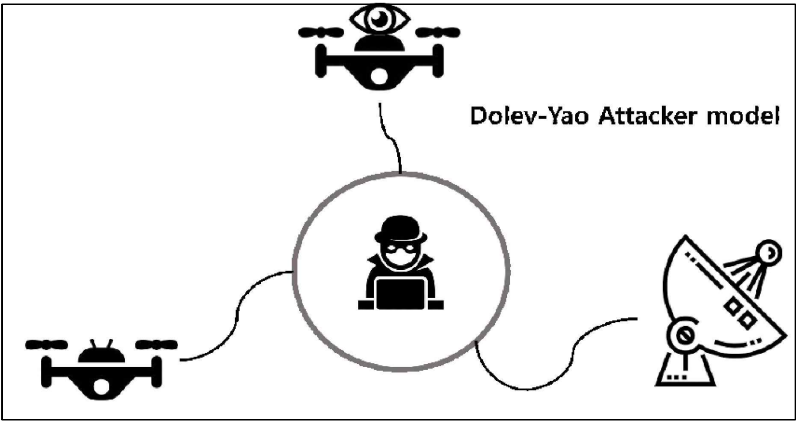
인터넷과 대부분의 다른 대형 컴퓨터 네트워크에는 서로 다른 참여 엔터티가 서로 연결하고 통신 할 수 있는 개방된 환경이 있다. 이는 일부 엔터티가 악의적이며 네트워크 내에서 라우팅되는 메시지를 스누핑, 변경 또는 제거할 뿐만 아니라 합법적인 사용자에게 대한 서비스를 거부하는 전체 네트워크를 차단할 수 있는 권한을 가지므로 보안 문제가 발생한다.

따라서, 암호화 통신의 공격자 모델은 주로 <그림 3-4>와 같은 DY(Dolev-Yao)의 공격자 모델[33]에 의해 모델링된다. 본 연구는 DY공격자 모델을 따라 공격자는 강력한 공격자로서 안전하지 않은 공개 채널 및 네트워크를 통해 모든 무선 트래픽을 도청하거나 사칭, 수정, 삭제 그리고 삽입이 가능하다고 가정하였다.

물리적 공격은 범위를 벗어나지만, 공격자가 실제 물리적인 드론 캡처를 통해 하드웨어에 손을 대면 비밀 키가 비교적 쉽게 추출될 수 있으므로 보안 키가 손상되었다고 가정한다.

이를 고려하여 제안된 프로토콜은 인접한 드론을 감시하기 위한 모니터링 드론

으로써 DY를 이용하여 모델링되고, 마찬가지로 드론들과 GCS 간 통신도 DY를 이용하여 모델링된다. GCS만이 완전히 신뢰되는 것으로 가정한다.



<그림 3-4> DY 위협 모델

본 연구에서 제시한 보안 요구사항은 <표3-3>과 같다.

<표 3-3> 본 연구의 보안 요구사항

구 분	내 용
기밀성	메시지의 기밀성을 제공하여 암호화된 통신을 지원해야한다.
무결성	메시지의 무결성을 제공하여 암호화된 통신을 지원해야한다.
상호인증	암호화된 전자서명을 이용하여 상대방에 대한 인증이 필요하다.
강력한 키교환	캡처를 대응하기 위해 PFS 및 PBS를 지원할 수 있는 강력한 키교환이 필요하다.
부인봉쇄	군이라는 특수한 환경에서는 임무를 하달할 때 항상 책임자가 필요하다. 본 연구에서는 군수분야 UAV를 대상으로 하였기 때문에 부인봉쇄가 필수적으로 필요하다.
서비스 거부 공격 대응	각 피어가 자원고갈형 서비스거부 공격에 의해 무력화가 되어서는 안된다
중간자 공격 대응	각 피어 간 통신 시 제 3자에 의한 참여가 불가능해야한다.

다음 <표 3-4>는 본 프로토콜에서 사용되는 용어에 대한 정리다.

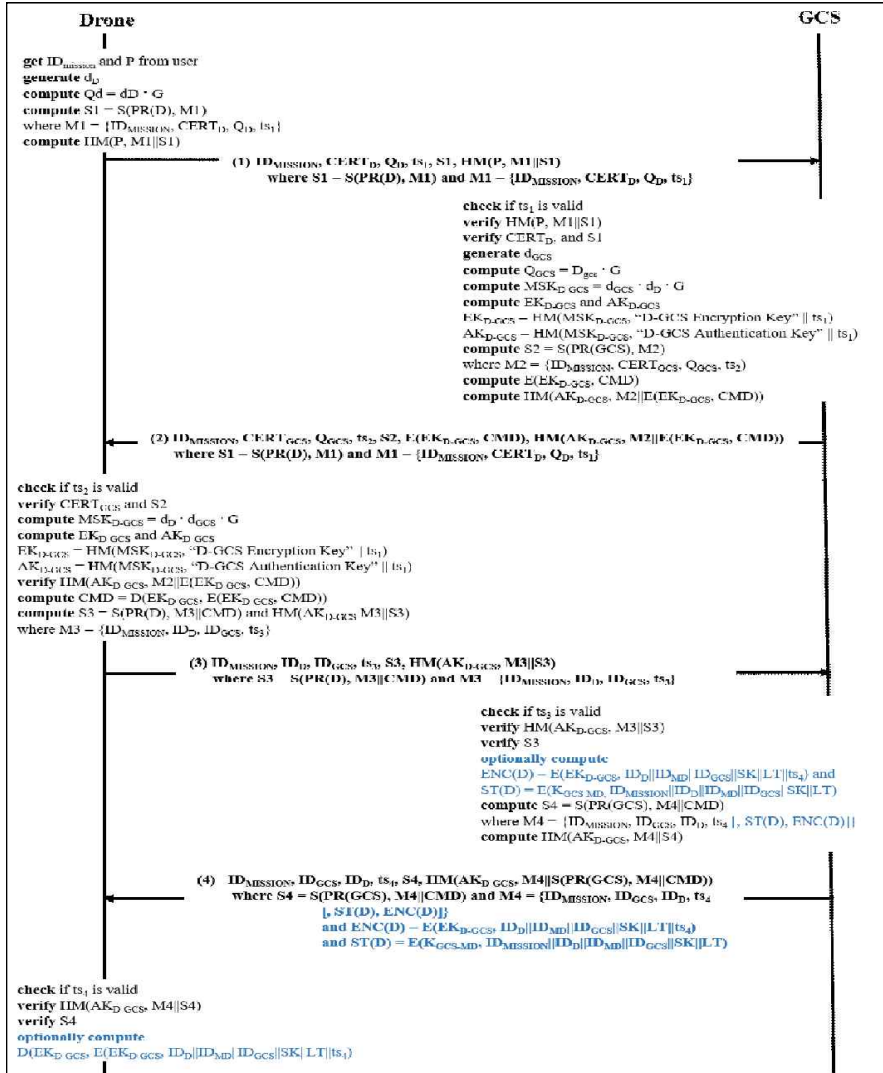
<표 3-4> 3장 용어 정리

용 어	설 명
IDMISSION	작전 아이디
P	핀 번호
G	타원곡선의 기준점
dD	ECDH 개인키
QD	ECDH 공개키 : $dD \bullet G$
PR	ECDAS 개인키
HM	HMAC값에 의해 보호된 메시지
S	전자서명
PR	개인키
MSK	마스터세션키
EK	암호화키
AK	인증키
CERT	인증서
ts	타임 스탬프
CMD	작전명령
SK	세션 키
ST	인증티켓
LT	키 생명 주기
ENC	암호화 형태

제 2 절 제안 프로토콜 설계

1. D-GCS 보안 프로토콜

다음의 <그림 3-5>는 UAV가 GCS에 직접적으로 통신을 하는 환경에서 적용이 가능한 보안 프로토콜이다.



<그림 3-5> D-GCS 보안 프로토콜

제안된 프로토콜의 순서는 다음과 같다.

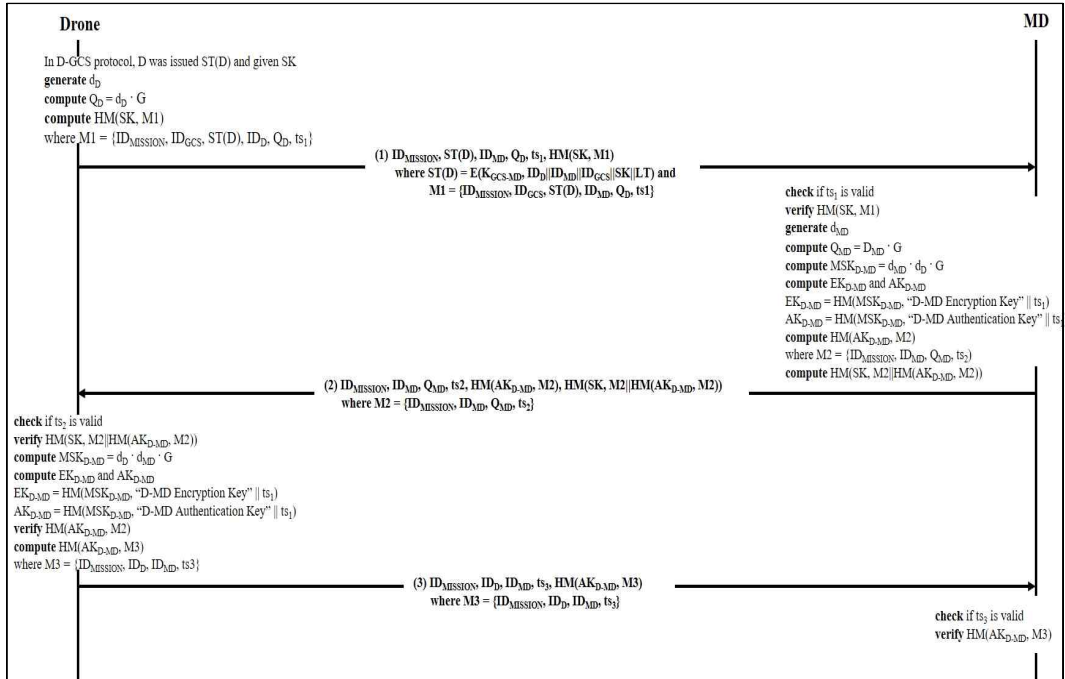
- (1) UAV는 U로부터 작전 아이디 $ID_{MISSION}$ 과 핀번호 P 를 입력받는다. 이후, ECDH 개인키 및 공개키 d_U 와 $d_U \bullet G$ 를 생성하고, $M1=(ID_{MISSION}, CERT(D), Q_D, ts1)$ 을 ECDAS 개인키 $PR(D)$ 로 전자서명한다. 그리고 $M1$ 및 전자서명 $S1$ 은 핀번호 P 를 통해 생성된 HMAC값에 의해 보호된다.
- (2) 메시지가 도착하면 GCS는 연산비용이 큰 공개키 연산이전에 타임스탬프 $ts1$ 을 검사하고, 핀번호 P 를 통해 HMAC값을 검증함으로써 자원고갈형 서비스거부공격에 대응한다. HMAC값이 타당하면 GCS는 UAV의 인증서 $CERT(D)$ 의 정당성을 검사하고, 전자서명 $S1$ 을 검증한다. $S1$ 이 유효하면 GCS는 UAV를 성공적으로 인증한다. UAV와 마찬가지로 GCS의 ECDH 개인키 및 공개키 d_{GCS} 와 $d_{GCS} \bullet G$ 를 연산한 후, d_{GCS} 와 UAV 공개키 Q_D 를 통해 세션 암호화 키 및 인증키 EK_{D-GCS} 와 AK_{D-GCS} 를 생성한다. 또한, (1)단계와 동일하게 $M2=(ID_{MISSION}, CERT(GCS), Q_{GCS}, ts2)$ 에 대한 전자서명 $S2$ 를 생성한다. (2)단계의 마지막과정으로 GCS는 EK_{D-GCS} 를 통해 작전명령 CMD 를 암호화하고, AK_{D-GCS} 를 적용하여 암호화 결과와 $M2$ 에 대한 HMAC값을 생성한다.
- (3) 메시지가 도착하면 UAV는 타임스탬프 $ts2$ 및 전자서명 $S2$ 를 검증함으로써 GCS를 인증하고 GCS의 ECDH 공개키 Q_D 에 대한 신뢰성을 확보한다. UAV는 Q_D 에 대한 신뢰성을 바탕으로 EK_{D-GCS} 와 AK_{D-GCS} 를 생성하여 수신 메시지의

HMAC값을 검증하며 그 결과 생성된 두 개의 키가 올바르게 교환되었음을 확인한다. 또한, 작전명령 CMD의 무결성을 확인할 수 있다. HMAC값이 정당하면 UAV는 $E(EK_{D-GCS}, CMD)$ 를 복호화하여 작전명령 CMD를 확보한다. (3)단계의 마지막 과정으로 $M3=(ID_{MISSION}, ID_D, ID_{GCS}, ts3)$ 및 CMD에 대한 전자서명 $S3$ 를 생성하고, $S3$ 과 $M3$ 에 대한 HMAC값을 연산한다.

- (4) 메시지를 수신하면 GCS는 타임스탬프 $ts3$ 와 HMAC을 검증한 후, 전자서명 $S3$ 의 유효성을 확인한다. $S3$ 가 유효하다면 GCS는 UAV가 작전명령을 성공적으로 수신했고 이에 따라 작전에 임할 것임을 신뢰할 수 있다. 여기서 $S3$ 는 향후 UAV가 작전명령 CMD 수신을 부인할 경우 사용될 수 있음. 이와 유사하게 GCS는 전자서명 $S4$ 를 통해 UAV가 자신이 작전명령 CMD를 송신한 사실을 증명할 수 있도록 한다. 또한, 공개키 연산으로 인한 서비스거부공격의 위협에 대응하기 위해 $AKD-GCS$ 를 바탕으로 HMAC값을 연산한다. 여기서 만약 UAV_D 가 모니터링 UAV_{MD} 와의 통신을 해야 하는 시나리오라면 GCS는 MD와의 공유 비밀키 K_{GCS-MD} 를 통해 추가적으로 D의 인증티켓 $ST(D)$ 를 발행하고, 암호화 세션키 SK 를 EK_{D-GCS} 기반의 암호화된 형태 $ENC(D)$ 로 변환해야 한다. 최종적으로 UAV가 메시지를 수신하면 타임스탬프 $ts4$ 및 HMAC값, 전자서명 $S4$ 를 검증하여 GCS가 작전명령CMD를 송신했음을 증명하고 이에 대한 근거를 확보한다 만일 UAV_D 가 모니터링 UAV_{MD} 와 통신할 상황이라면 $ENC(D)$ 를 복호화함으로써 세션키 SK 와 키생명주기 LT 등을 획득한다.

2. D-MD 보안 프로토콜

다음의 <그림 3-6>은 UAV가 MD를 통해 GCS에 간접적으로 통신을 하는 환경에서 적용이 가능한 보안 프로토콜이다.



<그림 3-6> D-MD 보안 프로토콜

제안된 프로토콜의 순서는 다음과 같다.

(1) UAV는 D-GCS 보안 프로토콜로부터 티켓 ST(D) 및 세션키 SK를 발급받는다.

이후, ECDH 개인키 및 공개키 d_D 와 $d_D \cdot G$ 를 생성하고, $M1 = (ID_{MISSION}, CERT(D), Q_D, ts1)$ 을 세션키 SK를 통해 HMAC값을 통해 보호한다.

(2) 메시지가 도착하면 MD는 연산비용이 큰 공개키 연산 이전에 타임스탬프 $ts1$ 을

검사하고, 세션키 SK를 통해 HMAC값을 검증함으로써 자원고갈형 서비스 거부 공격에 대응한다. HMAC값이 타당하면 MD는 UAV와 마찬가지로 MD의 ECDH 개인키 및 공개키 d_{MD} 와 $d_{MD} \cdot G$ 를 연산한 후, d_{MD} 와 UAV 공개키 Q_D 를 통해 세션 암호화 키 및 인증키 E_{KD-MD} 와 A_{KD-MD} 를 생성한다. 또한, (1)단계와 동일하게 $M2=(ID_{MISSION}, ID_{MD}, Q_{MD}, ts2)$ 에 대한 HMAC값을 생성한다. (2)단계의 마지막과정으로 MD는 M2 및 M2에 대한 HMAC값을 다시 SK를 통해 HMAC값을 생성한다.

- (3) 메시지가 도착하면 UAV는 타임스탬프 $ts2$ 및 M2의 HMAC값을 검증함으로써 MD를 인증하고 MD의 ECDH 공개키 Q_{MD} 에 대한 신뢰성을 확보한다. UAV는 Q_{MD} 에 대한 신뢰성을 바탕으로 E_{KD-MD} 와 A_{KD-MD} 를 생성하여 수신 메시지의 HMAC값을 검증하며 그 결과 생성된 두 개의 키가 올바르게 교환되었음을 확인한다. HMAC값이 정당하면 UAV는 A_{KD-MD} 를 통해 $M3=(ID_{MISSION}, ID_D, ID_{MD}, ts3)$ 의 HMAC값을 연산한다. 마지막으로 메시지를 수신한 MD는 타임스탬프 $ts3$ 을 검사하고 유효하다면 HMAC 값을 검증하여 키가 올바르게 교환되었음을 확인한다.

제 4 장 제안 프로토콜 평가

제 1 절 Scyther를 이용한 정형화 검증

1. Scyther 개요

Scyther는 2007년에 Cas Cremers에 의해 개발되었고, 정형화 검증을 위한 자동화된 도구로 Command Line 도구와 python 스크립팅 인터페이스를 통합한 그래픽 사용자 인터페이스를 제공한다. Scyther는 프로토콜 및 응용 프로그램에 대한 검증, 표현, 분석, 명세 및 도출 기능을 제공한다. 특히, Scyther는 프로토콜 행위 클래스를 제공하여 용이하게 프로토콜의 보안성을 정형화 검증하여 보안의 문제점을 지적하며, SPDL(Security Protocol Description Language)[34] 언어를 활용하여 표현력이 뛰어나다. SPDL 언어는 C / JAVA와 유사한 구문을 기반으로 하며(대소문자를 구분한다.), 역할들을 일련의 이벤트로 정의하여 대부분 전송 및 수신을 나타내는 이벤트로 구성한다.

보안 속성은 Claim 이벤트로 모델링 할 수 있다. Claim(Principal, Claim, Parameter)으로 구성되는데, Principal은 사용자 이름, Claim은 보안 속성, Parameter는 보안 속성을 확인한다.

프로토콜 검증을 위하여 Scyther는 세 가지 방법으로 사용할 수 있다.

- Verification claim: 보안 속성을 검증하거나 위조한다.
- Automatic claims: 사용자가 보안 속성을 Claim 이벤트로 지정하지 않으면

Scyther는 자동으로 클레임을 생성하고 확인한다.

- Characterization: Scyther는 프로토콜을 분석하고 프로토콜 역할의 실행을 포함하는 모든 추적의 유한 표현을 제공하므로 각 프로토콜 역할을 특성화 할 수 있다.

Scyther는 검증에 실패할 시 이에 대한 공격 그래프를 생성하고 각 Claim에 대한 개별 공격 그래프를 나타낸다.

2. 제안 프로토콜 정형화 검증

제안하는 프로토콜을 검증하기 위해서는 SPDL 소스코드를¹⁾ 통해 프로토콜에 맞게 모델링해야 한다. 본 절에서는 본 연구에서 제안한 두 가지 프로토콜을 하나로 합쳐서 정형화 검증을 진행하였다. 엔터티는 GCS, D 그리고, MD로 구성된다. 각 엔터티는 보내기 및 받기를 통해 다른 엔터티와 통신하고 클레임을 통해 엔터티의 동작을 확인하고 프로토콜이 얼마나 안전한지 보여준다.

본 논문에서 검증에 사용된 클레임 이벤트는 <표 4-1>에 표시된 기능별로 분류 할 수 있으며 자세한 내용은 [35]에 설명되어 있다. 클레임 이벤트는 인증 및 보안 확인을

1) 부록에 소스코드를 첨부하였다.

지원하며 제안된 프로토콜의 엔터티가 정상적으로 동작함을 증명할 수 있다. 또한 프로토콜에 사용된 키가 안전하게 파생되고 교환되는지 확인할 수 있다.

<표 4-1> Claim 이벤트 설명

이벤트	보안 속성
Alive	Authentication
Nisynch	Authentication
Niagree	Authentication
Weakagree	Authentication
Commit	Authentication
Secret	Secrecy

본 프로토콜의 검증을 시작하기 전에 Scyther에서는 Diffie - Hellman 알고리즘을 자체적으로 지원하지 않기 때문에 <그림 4-1>과 같이 모델링을 하였다.

```

protocol @oracle(DH) {
  role DH{
    var x, y: Nonce;
    recv_IDH1(DH, DH, h(g(y),x));
    send_IDH2(DH, DH, h(g(x),y));
  }
}

```

<그림 4-1> DH 알고리즘 모델링

이후, <그림 4-2>와 같이 본 프로토콜의 참여자를 뜻하는 Agent인 GCS, D 그리고 MD를 선언한 뒤, 메시지를 보낼 때 사용되는 값, 암호화에 이용되는 함수 그리고 공개키

쌍을 정의한다. 마지막으로, MD와 GCS가 사전에 키를 공유하는 것을 macro 함수를 통해 kMD로 설정한다.

```
var D, GCS, MD : Agent;
const idMission, idD, idMD, idGCS, P : Nonce;
hashfunction HM, h, g;

const pkD, pkGCS : Function;
secret skD, skGCS : Function;

inversekeys (pkD, skD);
inversekeys (pkGCS, skGCS);

macro kMD = k(MD, GCS);
```

<그림 4-2> 전역변수 선언 구문

Scyther에서는 전역변수를 통해 프로토콜에 이용되는 요소를 정의한 뒤, <그림 4-3>과 같이 ‘도우미(help) 프로토콜’을 ‘@’기호를 통해 모델링한다. 이는 분석결과에는 영향을 주지 않지만, 출력 그래프를 렌더링할 때 이용된다. 지역변수를 통해 본 프로토콜을 진행하면서 필요한 요소를 선언한 뒤, 전체적인 메시지를 모델링한다.

```
protocol @executability(O) {
  role Q {
    secret ekGCS, akGCS, ekD, ekD2, akD, akD2, ekMD, akMD, SK : SessionKey;
    fresh certGCS, certD, CMD, ts1, ts2, ts3, ts4, ts5, ts6, ts7, dGCS, dD, dD2, dMD, qGCS, qD, qD2, qMD : Nonce;
    var S1, S2, S3, S4, stD, encD : Nonce;

    rcvJ101(O, O, idMission, certD, g(dD), ts1, S1, HM(P, idMission, certD, g(dD), ts1, S1));
    sendJ102(O, O, idMission, certD, qD, ts1, S1, HM(P, idMission, certD, qD, ts1, S1));

    rcvJ103(O, O, idMission, certGCS, g(dGCS), ts2, S2, {CMD}ekGCS, HM(akGCS, idMission, certGCS, g(dGCS), ts2, S2, {CMD}ekGCS));
    sendJ104(O, O, idMission, certGCS, gGCS, ts2, S2, {CMD}ekD, HM(akD, idMission, certGCS, gGCS, ts2, S2, {CMD}ekD));

    rcvJ105(O, O, idMission, idD, idGCS, ts3, S3, HM(akD, idMission, idD, idGCS, ts3, S3));
    sendJ106(O, O, idMission, idD, idGCS, ts3, S3, HM(akGCS, idMission, idD, idGCS, ts3, S3));

    rcvJ107(O, O, idMission, idGCS, idD, ts4, S4, encD, stD, HM(akGCS, idMission, idGCS, idD, ts4, encD, stD, S4));
    sendJ108(O, O, idMission, idGCS, idD, ts4, S4, encD, stD, HM(akD, idMission, idGCS, idD, ts4, encD, stD, S4));

    rcvJ109(O, O, idMission, stD, idD, g(dD2), ts5, HM(idMission, stD, idD, g(dD2), ts5));
    sendJ110(O, O, idMission, stD, idD, qD2, ts5, HM(idMission, stD, idD, qD2, ts5));

    rcvJ111(O, O, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD, ts6), HM(SK, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD, ts6)));
    sendJ112(O, O, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD, g(dMD), ts6), HM(SK, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD, g(dMD), ts6)));

    rcvJ113(O, O, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD, ts7));
    sendJ114(O, O, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD, ts7));
  }
}
```

<그림 4-3> 도우미(help) 프로토콜 모델링

도우미 프로토콜에 대한 모델링이 끝나면 실제로 인증에 이용되는 프로토콜에 대한

상세 모델링이 필요하다. <그림 4-4>는 D에 대한 모델링이며 role 함수를 통해 선언한다. 지역변수를 통해 인증서, 타임스탬프 그리고 키와 같은 메시징에 이용되는 요소들을 선언한 뒤 macro함수를 통해 전자서명과 MSK에서 EK와 AK가 파생되는 과정을 설정하고 이후 D-MD에 이용되는 선택옵션에 대한 티켓을 선언한다. 이후 이에 대한 메시지 과정을 send와 recv을 통해 모델링을 진행한 뒤, 앞서 설명한 claim함수를 통해 본 프로토콜에서 검증해야 하는 요소들을 설정한다. 본 파트에서는 작전명령인 CMD와 MSK 그리고 MSK에서 파생되는 EK 및 AK에 대한 비밀성을 확인하고 Commit을 통해 올바르게 GCS와 합의가 되었는지에 대해 검증을 설정한다. 그리고 Alive, Nisynch, Niagree 및 Weakagree을 통해 D가 올바른 역할을 진행하고 있고 전달이 제대로 진행되는지에 대해 claim을 설정한다.

```

protocol MDSP(D, GCS, MD) {
  role D {
    fresh certD, dD, dD2 : Nonce;
    fresh ts1, ts3, ts5, ts7 : Nonce;
    var qGCS, CMD, certGCS, LT, qMD, ts2, ts4, ts6 : Nonce;
    var stD : Ticket;
    var SK : SessionKey;

    macro S1D = {idMission, certD, g(dD), ts1}skD;
    macro S2D = {idMission, certGCS, qGCS, ts2}skGCS;
    macro S3D = {idMission, idD, idGCS, ts3}skD;

    macro mskD = g(qGCS, dD);
    macro mskD2 = g(qMD, dD2);

    macro ekD = HM(mskD, ts1);
    macro ekD2 = HM(mskD2, ts5);

    macro akD = HM(mskD, ts1);
    macro akD2 = HM(mskD2, ts5);

    macro encD = {idD, idMD, idGCS, SK, LT, ts4}ekD;
    macro S4D = {idMission, idGCS, idD, ts4, encD, stD}skGCS;

    send_11(D, GCS, idMission, certD, g(dD), ts1, S1D, HM(P, idMission, certD, g(dD), ts1, S1D));
    recv_12(GCS, D, idMission, certGCS, qGCS, ts2, S2D, {CMD}ekD, HM(akD, idMission, certGCS, qGCS, ts2, S2D, {CMD}ekD));
    claimMD_Running, GCS, HM(akD, ts3, S3D);
    send_13(D, GCS, idMission, idD, idGCS, ts3, S3D, HM(akD, idMission, idD, idGCS, ts3, S3D));
    recv_14(GCS, D, idMission, idGCS, idD, ts4, S4D, encD, stD, HM(akD, idMission, idGCS, idD, ts4, encD, stD, S4D));

    claim(D, Secret, CMD);
    claim(D, Secret, mskD);
    claim(D, Secret, ekD);
    claim(D, Secret, akD);
    claim(D, Secret, ekD2);
    claim(D, Commit, GCS, HM(akD, ts3, S3D));

    send_15(D, MD, idMission, stD, idD, g(dD2), ts5, HM(idMission, stD, idD, g(dD2), ts5));
    recv_16(MD, D, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD, ts6), HM(SK, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD, ts6)));
    claimMD_Running, MD, HM(akD2, ts7));
    send_17(D, MD, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD, ts7));

    claim(D, Alive);
    claim(D, Nisynch);
    claim(D, Niagree);
    claim(D, Weakagree);
    claim(D, Secret, mskD2);
    claim(D, Secret, akD2);
    claim(D, Secret, ekD2);
    claim(D, Commit, MD, HM(akD2, ts7));
  }
}

```

<그림 4-4> D 모델링

D에 대한 모델링이 끝나면 프로토콜에 D와 통신을 하는 GCS에 대한 상세 모델링이 필요하다. <그림 4-5>는 GCS에 대한 모델링이며 마찬가지로 role 함수를 통해 선언한다. 지역변수를 통해 인증서, 타임스탬프 그리고 키와 같은 메시징에 이용되는 요소들을 선언한 뒤 macro함수를 통해 전자서명과 MSK에서 EK와 AK가 파생되는 과정을 설정하고 이후 D-MD에 이용되는 선택옵션에 대한 티켓을 선언한다. 이후 이에 대한 메시지 과정을 send와 recv을 통해 모델링을 진행한 뒤, 앞서 설명한 claim함수를 통해 본 프로토콜에서 검증해야 하는 요소들을 설정한다. 본 파트에서는 작전명령인 CMD와 MSK 그리고 MSK에서 파생되는 EK 및 AK에 대한 비밀성을 확인하고 Commit을 통해 올바르게 D와 합의가 되었는지에 대해 검증을 설정한다. 그리고 Alive, Nisynch, Niagree 및 Weakagree을 통해 GCS가 올바른 역할을 진행하고 있고 전달이 제대로 진행되는지에 대해 claim을 설정한다.

```

role GCS{
  fresh certGCS, dGCS, ts2, ts4, LT: Nonce;
  var certD, qD, ts1, ts3: Nonce;
  secret CMD: Nonce;
  fresh SK: SessionKey;

  macro S1GCS = {idMission, certD, qD, ts1}skD;
  macro S2GCS = {idMission, certGCS, g(dGCS), ts2}skGCS;
  macro S3GCS = {idMission, idD, idGCS, ts3}skD;

  macro mskGCS = g(qD, dGCS);
  macro ekGCS = HM(mskGCS, ts1);
  macro akGCS = HM(mskGCS, ts1);

  macro stD = {idMission, idD, idMD, idGCS, SK, LT}kMD;
  macro encD = {idD, idMD, idGCS, SK, LT, ts4}ekGCS;
  macro S4GCS = {idMission, idGCS, idD, ts4, encD, stD}skGCS;

  recv_I1(D, GCS, idMission, certD, qD, ts1, S1GCS, HM(P, idMission, certD, qD, ts1, S1GCS));
  send_I2(GCS, D, idMission, certGCS, g(dGCS), ts2, S2GCS, {CMD}ekGCS, HM(akGCS, idMission, idD, idGCS, ts3, S3GCS));
  recv_I3(D, GCS, idMission, idD, idGCS, ts3, S3GCS, HM(akGCS, idMission, idD, idGCS, ts3, S3GCS));
  claim(GCS, Running, D, HM(akGCS, ts4, S4GCS));
  send_I4(GCS, D, idMission, idGCS, idD, ts4, S4GCS, encD, stD, HM(akGCS, idMission, idD, ts4, encD, stD, S4GCS));

  claim(GCS, Alive);
  claim(GCS, Nisynch);
  claim(GCS, Niagree);
  claim(GCS, Weakagree);
  claim(GCS, Secret, CMD);
  claim(GCS, Secret, mskGCS);
  claim(GCS, Secret, akGCS);
  claim(GCS, Secret, ekGCS);
  claim(GCS, Commit, D, HM(akGCS, ts4, S4GCS));
}

```

<그림 4-5> GCS 모델링

본 연구에서는 선택적으로 D-MD에 대한 인증 과정을 포함하고 있으므로 본 절에서는 이 과정도 포함하여 검증을 진행하였다. GCS에 대한 모델링이 끝나면 마지막으로 MD에 대한 상세 모델링이 필요하다. <그림 4-6>은 MD에 대한 모델링이며 마찬가지로 role 함수를 통해 선언한다. 지역변수를 통해 인증서, 타임스탬프 그리고 키와 같은 메시징에 이용되는 요소들을 선언한 뒤 macro함수를 통해 전자서명과 MSK에서 EK와 AK가 파생되는 과정을 설정하고 이후 D-MD 인증 과정에 이용되는 티켓을 선언한다. 이후 이에 대한 메시지 과정을 send와 recv을 통해 모델링을 진행한 뒤, 앞서 설명한 claim함수를 통해 본 프로토콜에서 검증해야 하는 요소들을 설정한다. 본 파트에서는 MSK 그리고 MSK에서 파생되는 EK 및 AK에 대한 비밀성을 확인하고 Commit을 통해 올바르게 D와 합의가 되었는지에 대해 검증을 설정한다. 그리고 Alive, Nisynch, Niagree 및 Weakagree을 통해 MD가 올바른 역할을 진행하고 있고 전달이 제대로 진행되는지에 대해 claim을 설정한다.

```

role MD{
  fresh dMD, ts6 : Nonce;
  var qD2, LT, qD2, ts5, ts7 : Nonce;
  var SK : SessionKey;

  macro stD = {idMission, idD, idMD, idGCS, SK, LT}kMD;

  macro mskMD = g(qD2, dMD);
  macro akMD = HM(mskMD, ts5);
  macro ekMD = HM(mskMD, ts5);

  recv_15(D, MD, idMission, stD, idD, qD2, ts5, HM(idMission, stD, idD, qD2, ts5));
  claim(MD, Running, D, HM(akMD, ts5));
  send_16(MD, D, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD, g(dMD), ts6, HM(SK, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD, g(dMD), ts6)));
  recv_17(D, MD, idMission, idD, idMD, ts7, HM(akMD, idMission, idD, idMD, ts7));

  claim(MD, Alive);
  claim(MD, Nisynch);
  claim(MD, Niagree);
  claim(MD, Weakagree);
  claim(MD, Secret, mskMD);
  claim(MD, Secret, akMD);
  claim(MD, Secret, ekMD);
  claim(MD, Secret, D, HM(akMD, ts6));
}

```

<그림 4-6> MD 모델링

다음의 <그림 4-7>은 Scyther를 통해 본 프로토콜을 검증한 결과에 대한 것이다.

앞서 설명한 <표 4-1>의 Claim 이벤트를 각 엔티티가 모두 만족함을 볼 수 있다.

만약 실패를 한다면 Status 항목에 Fail이 뜨며 공격 시나리오를 도식화하여 보여준다.

Claim				Status	Comments
MUSP	D	MUSP,D2	Secret CMD	Ok	No attacks within bounds.
		MUSP,D3	Secret $g(qGCS,dD)$	Ok	No attacks within bounds.
		MUSP,D4	Secret $HM(g(qGCS,dD),ts1)$	Ok	No attacks within bounds.
		MUSP,D5	Secret $HM(g(qGCS,dD),ts1)$	Ok	No attacks within bounds.
		MUSP,D6	Commit $GCS, HM(HM(g(qGCS,dD),ts1),ts3,skD(idMission...$	Ok	No attacks within bounds.
		MUSP,D8	Alive	Ok	No attacks within bounds.
		MUSP,D9	Nisynch	Ok	No attacks within bounds.
		MUSP,D10	Niagree	Ok	No attacks within bounds.
		MUSP,D11	Weakagree	Ok	No attacks within bounds.
		MUSP,D12	Secret $g(qMD,dD2)$	Ok	No attacks within bounds.
		MUSP,D13	Secret $HM(g(qMD,dD2),ts5)$	Ok	No attacks within bounds.
		MUSP,D14	Secret $HM(g(qMD,dD2),ts5)$	Ok	No attacks within bounds.
		MUSP,D15	Commit MD, $HM(HM(g(qMD,dD2),ts5),ts7)$	Ok	No attacks within bounds.
	GCS	MUSP,GCS2	Alive	Ok	No attacks within bounds.
		MUSP,GCS3	Nisynch	Ok	No attacks within bounds.
		MUSP,GCS4	Niagree	Ok	No attacks within bounds.
		MUSP,GCS5	Weakagree	Ok	No attacks within bounds.
		MUSP,GCS6	Secret CMD	Ok	No attacks within bounds.
		MUSP,GCS7	Secret $g(qD,dGCS)$	Ok	No attacks within bounds.
		MUSP,GCS8	Secret $HM(g(qD,dGCS),ts1)$	Ok	No attacks within bounds.
		MUSP,GCS9	Secret $HM(g(qD,dGCS),ts1)$	Ok	No attacks within bounds.
		MUSP,GCS10	Commit D, $HM(HM(g(qD,dGCS),ts1),ts4,skGCS(idMission...$	Ok	No attacks within bounds.
	MD	MUSP,MD2	Alive	Ok	No attacks within bounds.
		MUSP,MD3	Nisynch	Ok	No attacks within bounds.
		MUSP,MD4	Niagree	Ok	No attacks within bounds.
		MUSP,MD5	Weakagree	Ok	No attacks within bounds.
		MUSP,MD6	Secret $g(qD2,dMD)$	Ok	No attacks within bounds.
		MUSP,MD7	Secret $HM(g(qD2,dMD),ts5)$	Ok	No attacks within bounds.
		MUSP,MD8	Secret $HM(g(qD2,dMD),ts5)$	Ok	No attacks within bounds.
		MUSP,MD9	Commit D, $HM(HM(g(qD2,dMD),ts5),ts6)$	Ok	No attacks within bounds.

<그림 4-7> Scyther verify result

제 2 절 BAN 논리를 이용한 정형화 검증

1. BAN 논리 개요

BAN 논리는 Michael Burrows, Martin Abadi, Roger Needham에 의해 제안된 기법으로 인증 프로토콜을 정형화된 방식으로 기술하고 분석하는 양상논리이다[24]. 다음 <표 4-2>와 <표 4-3>은 BAN Logic을 통해 정형화 검증을 수행하기 위한 표현 방법과 규칙에 대한 정리다.

<표 4-2> BAN 논리 표현

표 현	의 미
$P \text{ believes } X$	P는 X를 참이라고 믿는다.
$P \text{ sees } X$	P는 어떠한 순간에 X를 포함한 메시지를 받았다.
$P \text{ said } X$	P는 어떠한 순간에 X를 보냈다. 또한 P는 X를 믿는다.
$P \text{ controls } X$	P는 X에 대한 전권을 가진다.
$\text{Fresh}(X)$	X는 fresh하다.
$P \stackrel{K}{\longleftrightarrow} Q$	P와 Q가 비밀키 K를 공유하고 있다.
$\stackrel{K}{\longrightarrow} P$	P는 공개키 K를 가지고 있다. 그리고 P의 개인키는 K^{-1} 이다.
$P \stackrel{K}{\rightleftarrows} Q$	P와 Q가 공유비밀 K를 공유하고 있다.
$\{X\}_K$	X는 K로 암호화 되어있다.
X, Y	X와 Y가 결합 되어있다.

<표 4-3> BAN 논리 규칙

규 칙	수 식
MM: Message Meaning Rule	$\frac{P \text{ believes } Q \xleftrightarrow{K} P, P \text{ sees } \{X\}_K}{P \text{ believes } Q \text{ said } X}$
	$\frac{P \text{ believes } Q \xleftrightarrow{K} P, P \text{ sees } \langle X \rangle_K}{P \text{ believes } Q \text{ said } X}$
	$\frac{P \text{ believes } Q \xrightarrow{K}, P \text{ sees } \{X\}_{K^{-1}}}{P \text{ believes } Q \text{ said } X}$
NV: Nonce Verification Rule	$\frac{P \text{ believes } \text{fresh}(X), P \text{ believes } Q \text{ said } X}{P \text{ believes } Q \text{ believes } X}$
JR: Jurisdiction Rule	$\frac{P \text{ believes } Q \text{ controls } X, P \text{ believes } Q \text{ believes } X}{P \text{ believes } X}$
FR: Freshness Rule	$\frac{P \text{ believes } \text{fresh}(X)}{P \text{ believes } \text{fresh}(X, Y)}$
DR: Decomposition Rule	$\frac{P \text{ sees } (X, Y)}{P \text{ sees } X}$
BC: Belief Conjunction Rule	$\frac{P \text{ believes } X, P \text{ believes } Y}{P \text{ believes } (X, Y)}$
	$\frac{P \text{ believes } Q \text{ believes } (X, Y)}{P \text{ believes } Q \text{ believes } X}$
	$\frac{P \text{ believes } Q \text{ said } (X, Y)}{P \text{ believes } Q \text{ said } X}$
DH: Diffie-Helman Rule	$\frac{P \text{ believes } Q \text{ said } \xrightarrow{g^Y} Q, P \text{ believes } \xrightarrow{g^X} P}{P \text{ believes } P \xleftrightarrow{g^{XY}} Q}$
	$\frac{P \text{ believes } Q \text{ said } \xrightarrow{g^Y} Q, P \text{ believes } \xrightarrow{g^X} P}{P \text{ believes } P \xleftrightarrow{g^{XY}} Q}$

BAN 논리를 통한 정형화 검증은 3단계로 진행된다. 1단계에서 Idealization를 통해 프로토콜을 Logic에 해당되도록 모델링을 한 뒤, 2단계에서 Assumption을 통해 목표를 유도하기 위해 프로토콜의 진행 과정을 나타낸다. 마지막으로 3단계에서는 Derivation을 통해 프로토콜을 검증한다.

2. 제안 프로토콜 정형화 검증

BAN 논리를 통해 프로토콜을 검증하기 위해서는 Idealization을 통해 프로토콜을 논리에 맞게 모델링하고, Assumption을 통해 목표를 유도하기 위한 프로토콜의 가정을 표현한다. 마지막으로 Derivation을 통해 프로토콜의 목표인 Goal을 유도하여 검증한다. 이를 위해 <표 4-2>와 <표 4-3>의 BAN 논리의 표현 및 추론 규칙에 따라 제안하는 프로토콜을 모델링해야 한다. 제안된 프로토콜 중 D-GCS의 Idealization, Assumption은 <그림 4-8>, <그림 4-9>와 같다.

$$\begin{array}{l}
 (1) D \rightarrow GCS: \langle M_1, \{M_1\}_{PU(D^{-1})} \rangle_P \text{ where } M_1 = \langle ID_{MISSION}, g^X, ts_1 \rangle \\
 (2) GCS \rightarrow D: \langle M_2, \{M_2\}_{PU(GCS^{-1})} \rangle_{g^{XY}} \text{ where } M_2 = \langle ID_{MISSION}, g^Y, ts_2 \rangle \\
 (3) D \rightarrow GCS: \langle M_3, \{M_3, CMD\}_{PU(D^{-1})} \rangle_{g^{XY}} \text{ where } M_3 = \langle ID_{MISSION}, ID_D, ID_{GCS}, ts_3 \rangle \\
 (4) GCS \rightarrow D: \langle M_4, \{M_4, CMD\}_{PU(GCS^{-1})} \rangle_{g^{XY}} \text{ where } M_4 = \langle ID_{MISSION}, ID_{GCS}, ID_D, ts_4, ST(D), \{ID_{MISSION}, ID_{GCS}, ID_D, LT, ts_4, D \xrightarrow{SK} MD, \\
 D \xrightarrow{SK} MD\}_{g^{XY}} \rangle
 \end{array}$$

<그림 4-8> D-GCS 프로토콜 Idealization

(A1) $GCS \text{ believes } GCS \xrightarrow{P} D$	
(A2) $GCS \text{ believes } \text{fresh}(ts_1)$	
(A3) $GCS \text{ believes } \xrightarrow{PU(D)} D$	(A10) $GCS \text{ believes } \text{fresh}(ts_3)$
(A4) $GCS \text{ believes } \xrightarrow{g^T} GCS$	(A11) $D \text{ believes } \text{fresh}(ts_4)$
(A5) $D \text{ believes } \xrightarrow{PU(GCS)} GCS$	(A12) $D \text{ believes } GCS \text{ controls } D \xrightarrow{SK} MD$
(A6) $D \text{ believes } \text{fresh}(ts_2)$	(A13) $D \text{ believes } GCS \text{ controls } D \xrightarrow{SK} MD$
(A7) $D \text{ believes } \xrightarrow{g^T} D$	
(A8) $D \text{ believes } \text{fresh}(g^{XY})$	
(A9) $D \text{ believes } GCS \text{ controls } CMD$	

<그림 4-9> D-GCS 프로토콜 Assumption

본 검증에서는 평문 M1, M2, M3 그리고 M4는 기여가 없기 때문에 생략하였다. 또한, 전자서명 S1, S2, S3 그리고 S4는 동반하는 HMAC에도 포함되기 때문에 분석에 의한 중복성 제거를 위해 HMAC 밖에 있는 전자서명은 생략하였다. 마지막으로, EK_{D-GCS} 와 AK_{D-GCS} 는 g^{XY} 로부터 파생되었다고 모델링할 수 있기 때문에 각각 $GCS \xrightarrow{g^{XY}} D$ 및 $GCS \xleftarrow{g^{XY}} D$ 로 표현하였다.

제안된 프로토콜을 <그림 4-2>와 같이 BAN 논리로 검증하기 위해 앞서 설명한 표현 및 수식으로 표현한 것이다. 다음으로, <그림 4-9>와 같이 프로토콜의 정형화 검증 과정에서 목표를 유도하기 위해 활용되는 가정을 제안한다. 여기서 만약 프로토콜이 안전하다면 Derivation 유도 과정에서 프로토콜의 목적인 Goal을 유도하기 위한 추가적으로 필요한 Assumption이 요구되지 않는다.

다음의 수식은 위의 D-GCS 프로토콜의 Idealization 및 Assumption에 대한 유도 과정이다. 본 연구에서 필요로 하는 보안 요구사항에 대한 Goal을 만족하는 근거에는 빨간색으로 표시하였다.

<Derivation>

From (1)

(D1) *GCS sees* $M_1, \{M_1\}_{PU(D)^{-1}}_P$ by (1)

(D2) *GCS believes D said* $[M_1, \{M_1\}_{PU(D)^{-1}}]$ by (D1),(A1),MM

(D3) *GCS believes D believes* $[M_1, \{M_1\}_{PU(D)^{-1}}]$ by (D2),(A2),FR,NV

(D4) *GCS believes D said* $\{M_1\}_{PU(D)^{-1}}$ by (D2),BC

(D5) *GCS sees* $\{M_1\}_{PU(D)^{-1}}$ by (D4)

(D6) *GCS believes D said* M_1 by (D5),(A3),NV

(D7) *GCS believes D believes* M_1 by (D6),(A2),FR,NV

(D8) *GCS believes GCS* $\xrightarrow{g^{XY}} D$ by (D6),BC,(A4),DH

(D9) *GCS believes GCS* $\xrightarrow{g^{XY}} D$ by (D6),BC,(A4),DH

From (2)

(D10) *D sees* $\{M_2\}_{PU(GCS)^{-1}}, M_2, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D, \{CMD, g^{XY}\}_{g^{XY}} \rangle_{g^{XY}}$
by (2)

(D11) *D sees* $\{M_2\}_{PU(GCS)^{-1}}$ by (D10),DR

(D12) *D believes GCS said* M_2 by (D11),(A5),MM

(D13) *D believes GCS believes M_2 by (D12),(A6),FR,NV*

(D14) *D believes GCS $\xrightarrow{g^{XY}} D$ by (D12),BC,(A7),DH*

(D15) *D believes GCS $\xrightarrow{g^{XY}} D$ by (D12),BC,(A7),DH*

(D16) *D sees $M_2, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D, \{CMD, g^{XY}\}_{g^{XY}} \rangle_{g^{XY}}$ by (D10),DR*

(D17) *D believes GCS said $\left[M_2, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D, \{CMD, g^{XY}\}_{g^{XY}} \right]$
by (D16),(D15),MM*

(D18) *D believes GCS believes $\left[M_2, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D, \{CMD, g^{XY}\}_{g^{XY}} \right]$
by (D17),(A6),FR,NV*

(D19) *D believes GCS believes GCS $\xrightarrow{g^{XY}} D$ by (D18),BC*

(D20) *D believes GCS believes GCS $\xrightarrow{g^{XY}} D$ by (D18),BC*

(D21) *D believes GCS said $\{CMD, g^{XY}\}_{g^{XY}}$ by (D17),BC*

(D22) *D believes GCS believes $[CMD, g^{XY}]$ by (D21),(D14),MM,(A8),FR,NV*

(D23) *D believes CMD by (D22),BC,(A9),JR*

From (3)

(D23) *GCS sees $\left\langle M_3, \{M_3, CMD\}_{PU(D)^{-1}}, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D \right\rangle_{g^{XY}}$ by (3)*

(D24) *GCS believes D said $\left[M_3, \{M_3, CMD\}_{PU(D)^{-1}}, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} DRIGHT \right]$
by (D23),(D9),MM*

(D25) *GCS believes D believes $\left[M_3, \{M_3, CMD\}_{PU(D)^{-1}}, GCS \xrightarrow{g^{XY}} D, GCS \xrightarrow{g^{XY}} D \right]$
by (D24),(A10),FR,NV*

(D26) *GCS believes D believes GCS $\xrightarrow{g^{XY}} D$ by (D25),BC*

(D27) *GCS believes D believes GCS $\xleftrightarrow{g^{xy}} D$ by (D25), BC*

(D28) *GCS sees $\{M_3, CMD\}_{PU(D)^{-1}}$ by (D24), BC*

(D29) *GCS believes D believes $[M_3, CMD]$ by (D28), (A3), MM, (A10), FR, NV*

(D30) *GCS believes D believes CMD by (D29), BC*

From (4)

(D31) *D sees $M_4, \{M_4, CMD\}_{PU(GCS)^{-1}}$ by (4)*

(D32) *D believes GCS said $[M_4, \{M_4, CMD\}_{PU(GCS)^{-1}}]$ by (D31), (D15), MM*

(D33) *D believes GCS believes $[M_4, \{M_4, CMD\}_{PU(GCS)^{-1}}]$*

by (D32), (A11), FR, NV

(D34) *D sees $\{M_4, CMD\}_{PU(GCS)^{-1}}$ by (D32), BC*

(D35) *D believes GCS believes $[M_4, CMD]$ by (D34), A(5), MM, (A11), FR, NV*

(D36) *D believes CMD by (D35), BC, (A9), JR*

(D37) *D believes GCS believes $\left[ST(D), \left\{ID_{MISSION}, ID_{GCS}, ID_D, LT, ts_4, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD\right\}_{g^{xy}}\right]$*
by (D35), BC

(D38) *D believes GCS believes ST(D) by (D37), BC*

(D39) *D sees $\left\{ID_{MISSION}, ID_{GCS}, ID_D, LT, ts_4, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD\right\}_{g^{xy}}$ by (D37), BC*

(D40) *D believes GCS believes $\left[ID_{MISSION}, ID_{GCS}, ID_D, LT, ts_4, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD\right]$*
by (D39), (D14), MM, (A11), FR, NV

(D41) *D believes $D \xleftrightarrow{SK} MD$ by (D40), BC, (A12), JR*

(D42) *D believes $D \xleftrightarrow{SK} MD$ by (D40), BC, (A13), JR*

보조정리-1. D와 GCS는 상호인증을 한다.

증명. (D7)에 의해 GCS가 D를 인증함을 알 수 있고, (D13)에 의해 D가 GCS를 인증함을 알 수 있다. 따라서 D와 GCS는 상호인증을 한다.

보조정리-2. D와 GCS는 인증키 AK 및 암호화키 EK를 안전하게 교환한다.

증명. GCS의 경우, (D8) 그리고 (D9)를 통해 AK 및 EK를 직접 D로부터 전달받기 때문에 직접적인 믿음이 있다. D의 경우, (D14) 그리고 (D15)를 통해 AK 및 EK를 직접 GCS로부터 전달받기 때문에 직접적인 믿음이 있다. 또한, (D19), (D20), (D26) 그리고 (D27)을 통해 AK 및 EK에 대한 믿음을 신뢰함으로써 간접적인 믿음을 확보하였다. 따라서, AK 및 EK는 D와 GCS 사이에서 안전하게 교환되었다.

보조정리-3. D와 GCS는 D-GCS 시나리오에서 세션키 SK를 안전하게 교환한다.

증명. GCS는 SK를 직접 생성하기 때문에 이를 믿을 수 있다. D의 경우, (D41)과 (D42)를 통해 SK를 직접 GCS로부터 전달받기 때문에 SK에 대한 직접적인 믿음이 있다. 따라서 D와 GCS는 D-GCS 시나리오의 세션키 SK를 안전하게 교환한다.

보조정리-4. D와 GCS는 서비스거부 공격 대응을 지원한다.

증명. GCS의 경우, (D3)을 통해 M1이 Freshness함을 보였고 P를 알지 못하면 메시지를

발급하지 않기 때문에 서비스거부 공격에 대한 방어를 지원한다. 또한, (D25)를 통해 M3이 사전에 공유된 키인 g^{xy} 에서 파생된 EK 및 AK을 통해 암호화가 되어 있으므로 g^{xy} 를 알지 못하면 다음 메시지를 발급하지 않기 때문에 서비스거부 공격에 대한 방어를 지원한다.

D의 경우, 타임스탬프를 통해 M2이 Freshness함을 보였고 사전에 공유된 키인 g^{xy} 에서 파생된 EK 및 AK을 통해 암호화를 하기 때문에 서비스 거부 공격에 대한 방어를 지원한다.

또한, (D33)을 통해 M4가 사전에 공유된 키인 EK 및 AK에서 파생된 g^{xy} 을 통해 암호화가 되어 있으므로 g^{xy} 를 알지 못하면 다음 메시지를 발급하지 않기 때문에 서비스 거부 공격에 대한 방어를 지원한다. 따라서 D와 GCS는 서비스 거부 공격 방어를 지원한다.

보조정리-5. D와 GCS는 부인봉쇄를 지원한다.

증명. GCS의 경우, (D3)을 통한 D의 인증 요청 및 (D29)를 통해 작전명령 수신에 대한 검증을 확인한다. D의 경우, (D22)를 통한 D의 인증 요청 및 (D35)를 통한 작전명령 수신에 대한 검증을 확인한다. 따라서, D와 GCS는 부인봉쇄를 지원한다.

보조정리-6. D와 GCS는 작전명령 CMD에 대한 기밀성을 지원한다.

증명. GCS의 경우, (D23) 및 (D30)을 통해 작전명령인 CMD가 D가 믿고 있음을 검증할 수 있다. 또한, D의 경우, (D36)을 통해 작전명령인 CMD가 GCS가 보냈음을 검증할 수 있다. 따라서, D와 GCS는 작전명령 CMD에 대한 기밀성을 지원한다.

보조정리-7. D와 GCS는 인증키 AK 및 암호화키 EK에 대한 기밀성을 지원한다.

증명. GCS의 경우, (D8) 및 (D9)를 통해 안전하게 상호교환 됨을 확인하고, (D14) 그리고 (D15)를 통해 g^{xy} 가 제대로 파생되었음을 검증하여, 인증키 AK 및 암호화키 EK에 대한 기밀성을 확인할 수 있다. D의 경우, (D19) 및 (D20)을 통해 안전하게 상호교환 됨을 확인하고, (D26) 그리고 (D27)를 통해 g^{xy} 가 제대로 파생되었음을 검증하여, 인증키 AK 및 암호화키 EK에 대한 기밀성을 확인할 수 있다. 따라서, D와 GCS는 인증키 AK 및 암호화키 EK에 대한 기밀성을 지원한다.

보조정리-8. D와 GCS는 D-GCS 시나리오의 세션키 SK에 대한 기밀성을 지원한다.

증명. D의 경우, **보조정리-3**을 통해 키가 안전하게 교환됐음에 대한 직접적인 믿음을 확인하였고, 마찬가지로, (D41) 및 (D42)를 통해 SK에 대한 기밀성을 지원한다. 따라서, D와 GCS는 D-MD 세션키인 세션키 SK에 대한 기밀성을 지원한다.

보조정리-9. D와 GCS는 메시지에 대한 무결성 및 데이터 인증을 지원한다.

증명. GCS의 경우, (D7) 및 (D29)를 통해 D가 M1 그리고 M3을 믿음을 확인하여 메시지에 대한 무결성 및 데이터 인증을 지원함을 알 수 있다. D의 경우, (D13) 및 (D35)를 통해 GCS가 M2 그리고 M4을 믿음을 확인하여 메시지에 대한 무결성 및 데이터 인증을 지원함을 알 수 있다. 따라서, D와 GCS는 메시지에 대한 무결성

및 데이터 인증을 지원한다.

보조정리-10. D와 GCS는 중간자 공격에 대한 대응을 지원한다.

증명. D와 GCS는 인증서를 통해 ECDSA공개키의 정당성을 증명할 수 있도록 하고, ECDH 공개키는 전자서명을 통해 정당성을 증명할 수 있도록 한다. 또한, (D7)과 (D13)을 통해 ECDH 공개키를 신뢰할 수 있음을 확인할 수 있다. 따라서, D와 GCS는 중간자 공격에 대한 대응을 지원한다.

보조정리-11. D와 GCS는 PFS 및 PBS를 지원한다.

증명. 보조정리-2 및 보조정리-7을 통해 g^{XY} 가 D와 GCS 사이에 Diffie-Hellman 키 교환을 통해 안전하게 설정되었음을 보여준다. 개인키 X와 Y는 g^{XY} 가 어떤 경우에도 복구되지 않도록 두 당사자에서 즉시 제거되기 때문에 g^{XY} 에서 파생된 AK와 EK는 PFS 및 PBS를 지원한다.

정리-1. 본 논문에서 제안된 D-GCS 프로토콜은 보안 요구사항을 만족하며 공격에 안전하다.

증명. 보조정리-1부터 보조정리-11까지를 통해 제안된 D-GCS 프로토콜은 상호인증, 안전한 키 분배, 부인봉쇄, 기밀성, 무결성, 데이터 인증, PFS 및 PBS와 같은 보안

요구사항을 만족하며 중간자 공격 대응과 서비스 거부 공격 대응을 지원함으로써 공격에 안전하다.

본 논문에서 제안된 프로토콜 중 D-MD의 Idealization, Assumption은 <그림 4-10>, <그림 4-11>과 같다.

$$\begin{aligned}
 (1) \quad D \rightarrow MD: & \left[ST(D), \left\langle M_1, D \xrightarrow{SK} MD, D \xrightarrow{SK} MD \right\rangle_{SK} \right], \\
 & \text{where } M_1 = \{ID_{MISSION}, ID_{GCS}, ST(D), g^X, ts_1\} \text{ and} \\
 & SK(D) = \left\{ ID_{MISSION}, ID_{MD}, ID_{GCS}, D \xrightarrow{SK} MD, D \xrightarrow{SK} MD, LT \right\}_{K_{GCS-MD}} \\
 (2) \quad MD \rightarrow D: & \left\langle M_2, \left\langle M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right\rangle_{g^{XY}} \right\rangle_{SK}, \\
 & \text{where } M_2 = \{ID_{MISSION}, ID_{MD}, g^Y, ts_2\} \\
 (3) \quad D \rightarrow MD: & \left\langle M_3, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right\rangle_{g^{XY}}, \\
 & \text{where } M_3 = \{ID_{MISSION}, ID_D, ID_{MD}, ts_3\}
 \end{aligned}$$

<그림 4-10> D-MD 프로토콜 Idealization

(A1) $MD \text{ believes } GCS \xrightarrow{K_{GCS,MD}} MD$ (A2) $MD \text{ believes fresh}(LT)$ (A3) $MD \text{ believes } GCS \text{ controls } D \xrightarrow{SK} MD$ (A4) $MD \text{ believes } GCS \text{ controls } D \xrightarrow{SK} MD$ (A5) $MD \text{ believes fresh}(ts_1)$ (A6) $MD \text{ believes } \xrightarrow{g^Y} MD$ (A7) $D \text{ believes } D \xrightarrow{SK} MD$ (A8) $D \text{ believes fresh}(ts_2)$	(A9) $D \text{ believes } \xrightarrow{g^X} D$ (A10) $MD \text{ believes fresh}(ts_3)$
---	--

<그림 4-11> D-MD 프로토콜 Assumption

본 검증에서는 평문 M1, M2 그리고 M3는 분석에 기여하지 않았기 때문에 생략하였다. 또한, EK_{D-MD} 와 AK_{D-MD} 는 g^{XY} 로부터 파생되었다고 모델링할 수 있기 때문에 각각 $GCS \xrightarrow{g^{XY}} D$ 및 $GCS \xleftarrow{g^{XY}} D$ 로 표현하였다. 다음의 수식은 위의 D-MD 프로토콜의 Idealization 및 Assumption에 대한 유도 과정이다. 본 연구에서 필요로 하는 보안 요구사항에 대한 Goal을 만족하는 근거에는 빨간색으로 표시하였다.

<Derivation>

From (1)

$$(D1) \text{ MD sees } \left[ST(D), \quad M_1, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD \right]_{SK} \text{ by (1)}$$

$$(D2) \text{ MD sees } ST(D) \text{ by (D1), DR}$$

$$(D3) \text{ MD believes GCS believes } \left[ID_{MISSION}, ID_{MD}, ID_{GCS}, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD, LT \right] \text{ by (D2), (A1), MM, (A2), FR, NV}$$

$$(D4) \text{ MD believes GCS believes } D \xleftrightarrow{SK} MD \text{ by (D3), BC}$$

$$(D5) \text{ MD believes GCS believes } D \xleftrightarrow{SK} MD \text{ by (D3), BC}$$

$$(D6) \text{ MD believes } D \xleftrightarrow{SK} MD \text{ by (D4), (A3), JR}$$

$$(D7) \text{ MD believes } D \xleftrightarrow{SK} MD \text{ by (D5), (A4), JR}$$

$$(D8) \text{ MD sees } \left\langle M_1, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD \right\rangle_{SK} \text{ by (D1), DR}$$

$$(D9) \text{ MD believes } D \text{ said } \left[M_1, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD \right] \text{ by (D8), (D7), MM}$$

$$(D10) \text{ MD believes } D \text{ believes } \left[M_1, D \xleftrightarrow{SK} MD, D \xleftrightarrow{SK} MD \right] \text{ by (D9), (A5), FR, NV}$$

(D11) *MD believes D believes $D \xleftrightarrow{SK} MD$ by (D10), BC*

(D12) *MD believes D believes $D \xleftrightarrow{SK} MD$ by (D10), BC*

(D13) *MD believes $D \xrightarrow{g^{XY}} MD$ by (D9), BC, (A6), DH*

(D14) *MD believes $D \xrightarrow{g^{XY}} MD$ by (D9), BC, (A6), DH*

From (2)

(D15) *D sees $M_2, M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \Big\rangle_{g^{XY}} \Big\rangle_{SK}$ by (2)*

(D16) *D believes MD said $\left[M_2, \left\langle M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right\rangle_{g^{XY}} \text{RIGHT} \right]$
by (D15), (A7), MM*

(D17) *D believes MD believes $\left[M_2, \left\langle M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right\rangle_{g^{XY}} \text{RIGHT} \right]$
by (D16), (A8), FR, NV*

(D18) *D believes $D \xrightarrow{g^{XY}} MD$ by (D16), BC, (A9), DH*

(D19) *D believes $D \xrightarrow{g^{XY}} MD$ by (D16), BC, (A9), DH*

(D20) *D sees $\left\langle M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right\rangle_{g^{XY}}$ by (D16), BC*

(D21) *D believes MD believes $\left[M_2, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right]$
by (D20), (D19), MM, (A8), FR, NV*

(D22) *D believes MD believes $D \xrightarrow{g^{XY}} MD$ by (D21), BC*

(D23) *D believes MD believes $D \xrightarrow{g^{XY}} MD$ by (D21), BC*

From (3)

$$(D24) \quad MD \text{ sees } M_3, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \Bigg\rangle_{g^{XY}} \text{ by (3)}$$

$$(D25) \quad MD \text{ believes } D \text{ believes } \left[M_3, D \xrightarrow{g^{XY}} MD, D \xrightarrow{g^{XY}} MD \right] \\ \text{by (D24), (D14), MM, (A10), FR, NV}$$

$$(D26) \quad MD \text{ believes } D \text{ believes } D \xrightarrow{g^{XY}} MD \text{ by (D25), BC}$$

$$(D27) \quad MD \text{ believes } D \text{ believes } D \xrightarrow{g^{XY}} MD \text{ by (D25), BC}$$

보조정리-12. D와 MD는 상호인증을 한다.

증명. (D10)에 의해 MD가 D를 인증함을 알 수 있고, (D17)에 의해 D가 MD를 인증함을 알 수 있다. 따라서 D와 MD는 상호인증을 한다.

보조정리-13. D와 MD는 인증키 AK 및 암호화키 EK를 안전하게 교환한다.

증명. MD의 경우, (D13) 그리고 (D14)를 통해 AK 및 EK를 직접 D로부터 전달받은 MSK에서 유도되기 때문에 직접적인 믿음이 있다. D의 경우, (D18) 그리고 (D19)를 통해 AK 및 EK를 직접 GCS로부터 전달받은 MSK에서 유도되기 때문에 직접적인 믿음이 있다. 또한, (D22), (D23), (D26) 그리고 (D27)을 통해 AK 및 EK에 대한 믿음을 신뢰함으로써 간접적인 믿음을 확보하였다. 따라서, AK 및 EK는 D와 MD 사이에서 안전하게 교환한다.

보조정리-14. D와 MD는 서비스거부 공격 방어를 지원한다.

증명. MD의 경우, (D10)을 통해 M1이 Freshness함을 보였고 SK를 알지 못하면 메시지를 발급하지 않기 때문에 서비스거부 공격에 대한 방어를 지원한다. D의 경우, (D17)을 통해 M2가 사전에 공유된 키인 g^{xy} 을 통해 파생된 EK 및 AK을 이용하여 암호화가 되어 있으므로 g^{xy} 를 알지 못하면 다음 메시지를 발급하지 않기 때문에 서비스거부 공격에 대한 방어를 지원한다. 따라서, D와 MD는 서비스거부 공격 방어를 지원한다.

보조정리-15. D와 MD는 인증키 AK 및 암호화키 EK에 대한 기밀성을 지원한다.

증명. MD 경우, (D13) 및 (D14)를 통해 안전하게 상호교환 됨을 확인하고, g^{xy} 가 제대로 파생되었음을 검증하여, 인증키 AK 및 암호화키 EK에 대한 기밀성을 확인할 수 있다. D의 경우, (D18) 및 (D19)을 통해 안전하게 상호교환 됨을 확인하고, g^{xy} 가 제대로 파생되었음을 검증하여, MD가 파생된 키를 믿음을 확인 할 수 있다. 따라서, D와 MD는 인증키 AK 및 암호화키 EK에 대한 기밀성을 지원한다.

보조정리-16. D와 MD는 D-MD 시나리오의 세션키 SK에 대한 기밀성을 지원한다.

증명. MD의 경우, (A1)을 통해 (D6) 및 (D7)을 유도하여 SK를 GCS와 MD가 공유하고 있음을 확인할 수 있고, 이를 통해 경유하여 보냈음을 알 수 있다. 따라서,

D와 MD는 세션키 SK에 대한 기밀성을 지원한다.

보조정리-17. D와 MD는 메시지에 대한 무결성 및 데이터 인증을 지원한다.

증명. MD의 경우, (D10) 및 (D25)를 통해 D가 M1 그리고 M3을 믿음을 확인하여 메시지에 대한 무결성 및 데이터 인증을 지원함을 알 수 있다. D의 경우, (D17)을 통해 MD가 M2의 믿음을 확인하여 메시지에 대한 무결성 및 데이터 인증을 지원함을 알 수 있다. 따라서, D와 MD는 메시지에 대한 무결성 및 데이터 인증을 지원한다.

보조정리-18. D와 MD는 중간자 공격에 대한 대응을 지원한다.

증명. D와 MD는 ECDH 공개키를 전자서명을 통해 정당성을 증명할 수 있도록 한다. 또한, (D10)과 (D17)을 통해 ECDH 공개키를 신뢰할 수 있음을 확인할 수 있다. 따라서, D와 MD는 중간자 공격에 대한 방어를 지원한다.

보조정리-19. D와 MD는 PFS 및 PBS를 지원한다.

증명. 보조정리-13 및 보조정리-15를 통해 g^{XY} 가 M와 MD 사이에 Diffie-Hellman 키 교환을 통해 안전하게 설정되었음을 보여준다. 개인키 X와 Y는 g^{XY} 가 어떤 경우에도 복구되지 않도록 두 당사자에서 즉시 제거되기 때문에 g^{XY} 에서 파생된 AK 및 EK는 PFS 및 PBS를 지원한다.

정리-2. 본 논문에서 제안된 D-MD 프로토콜은 보안 요구사항을 만족하며 공격에 안전하다.

증명. 보조정리-12부터 보조정리-19까지를 통해 제안된 D-MD 프로토콜은 상호 인증, 안전한 키 분배, 기밀성, 무결성, 데이터 인증, PFS 및 PBS와 같은 보안 요구사항을 만족하며 중간자 공격 대응과 서비스 거부 공격 대응을 지원함으로써 공격에 안전하다.

제 3 절 제 안 프로 토 콜 비 교 및 성 능 분 석

1. 제 안 프로 토 콜 비 교

다음의 <표 4-4>는 2장에서 분석한 기존연구들과의 보안성 성능에 대한 비교표이다. Galios, Inc의 GEC 프로토콜은 PFS 및 PBS를 고려하였지만 과도한 연산이 필요하기 때문에 비행 중 키 업데이트에 큰 부담을 준다. 또한, 자원고갈형 서비스 거부 공격에 대한 대응이 부족하다.

Seo et al.의 DSD 프로토콜은 키를 보호하는 데 강력한 강점을 갖고 있다. 하지만, 연산 오버헤드가 과도하기 때문에 본 연구와는 맞지 않으며 암호화 모듈 배포문제가 있다.

Olivier Blazy et al.의 UAS 프로토콜은 해시체인 원리의 키 스트림을 이용한 키갱신을 통해 PFS 및 PBS를 충족하였지만, 초기인증 및 상호인증에 대한 언급이 없으며, 본 연구의 시나리오에서는 부인봉쇄에 대한 요구사항이 필수적으로 필요하기 때문에 적절하지 않다.

마지막으로, Wazid et al.의 WDKVR 프로토콜은 XOR 및 해시만을 이용하는 방안을 통해 경량화된 기법을 제안하였으나, 연산과정이 과도하게 복잡하고 사용자의 바이오 인증을 요구한다. 또한, 부인봉쇄, PFS 그리고 PBS를 지원하지 않으므로 본 논문의 시나리오와 맞지 않는다.

<표 4-4> 기존연구들과 제안 프로토콜의 보안 요구사항 비교표

	GEC 프로토콜 [12]	DSD 프로토콜 [17]	UAS 프로토콜 [18]	WDKVR 프로토콜 [19]	본 프로토콜
기밀성	○	○	○	○	○
무결성	○	○	○	○	○
상호인증	○	○	○	○	○
부인봉쇄	○	○	X	X	○
PFS	○	X	○	X	○
PBS	○	X	○	X	○
서비스거부 공격 대응	X	○	X	○	○
중간자 공격 대응	○	○	○	○	○
D2D 보안 지원	X	X	X	△	○
Note: ○: 지원함 X: 지원하지 않음 △: 지원하지만, 미흡함					

2. 제안 프로토콜 성능 분석

본 논문에서는 2장에서 설명한 기존의 프로토콜들과 제안한 기법의 연산 오버헤드를 측정하고 비교를 통해 효율성 검증에 진행하였다.

본 연구는 초기인증인 D-GCS 단계에서 군이라는 특수한 환경으로 인해 임무 하달에 대한 책임소재의 존재가 필요하고, 캡처 시 정보가 유출되는 것을 방지하기 위해 필수적으로 필요한 보안요소인 PFS 및 PBS를 지원하기 위해 비교적 연산 오버헤드가 높은 DH의 키교환 기법 및 전자서명의 사용이 불가피하였다.

하지만, 1장에서 설명한바와 같이 군수분야 UAV 환경에서는 군집드론에 적용할 수 있는 D2D 보안 통신이 핵심이라고 강조되고 있기 때문에 비행 중의 경량성이 매우 중요하다. 때문에, 기존의 초기인증인 D-GCS에서의 강력한 키교환을 진행하여 비행 중에 상호 인증 및 키교환을 진행해야하는 본 연구의 핵심인 D-MD 시나리오에서의 경량성을 지원한다. 군수분야 환경의 보안 요구사항에서 가장 중요한 PFS 및 PBS 그리고 부인봉쇄에 대해 지원하지 않으며, 각 드론마다 바이오인증을 해야하므로 이에 대한 부담이 매우 크다. 또한, UAS 프로토콜은 기존의 상호인증 및 초기설정 단계가 생략되어 있어 경량화된 키교환 기법을 분석하는데 의미가 있지만, 효율성을 비교하는데 무리가 있었다. DSD 프로토콜은 화이트박스 기법 이용을 통해 강력한 키보호를 지원하지만, 다른 기법에 비해 오버헤드가 매우 높다. 마지막으로 GEC 프로토콜은 ECDH 등 전자서명을 지원하여 강력한 보안성을 지원하지만 서비스거부 공격에 대한

면역이 부족하고 오버헤드가 매우 높기 때문에 비행 중 키교환을 통해 보안 채널을 형성해야하는 D2D 시나리오에 적용하는데 무리가 있다. 앞서 설명을 통한 비교분석으로 본 기법이 효율적으로 키교환 및 인증이 진행됨을 볼 수 있다.

다음의 <표 4-5>는 제안된 프로토콜과 기존의 연구들에 대한 연산 오버헤드에 대한 비교표이다.

<표 4-5> 기존연구들과 제안 프로토콜의 연산 오버헤드 비교표

제안 프로토콜	D-GCS	D-MD	Note: • C_{bio} : 바이오 인증 • C_{WBC}: 화이트 박스 암호화 기법 • C_{SC} : 대칭키 암호화 기법 • C_{PC} : 공개키 암호화 기법 • C_S : 전자서명 • C_{SV} : 서명 검증 • C_{DH} : Diffie-Hellman • C_{XoR} : XOR연산 • C_{HM} : HMAC • C_H : Hash • C_C : 인증서
	$7C_{SC} + 4C_S + 4C_{SV} + 2C_{DH} + 11C_{HM} + 2C_C$	$C_{SC} + 2C_{DH} + 8C_{HM}$	
WDKVR 프로토콜 [19]	초기단계	인증단계	
$C_{bio} + 8C_{XoR} + 12C_H$	$C_{bio} + 12C_{XoR} + 32C_H$		
UAS 프로토콜 [18]	초기단계	인증단계	
	No mentioned	$2C_{SC} + 3C_{XoR} + 3C_H$	
DSD 프로토콜 [17]	$5C_{WBC} + 2C_S + 2C_{SV}$		
GEC 프로토콜 [12]	$2C_{PC} + 2C_S + 2C_{SV}$		

제 5 장 결 론

UAV는 그동안 주로 정찰, 공격 등의 군수분야 목적으로 개발되어 활용되어 왔으나, 최근에는 IT기술의 발전, 드론 부품 가격의 하락 등으로 인하여 레저활동, 영상 촬영 등 민간용으로도 보급이 확대되고 있다. 세계 UAV 시장은 2020년 115억 달러 규모로 성장할 것으로 예측되며, 현재 전체 시장에서 민간용 UAV가 차지하는 비중은 21%으로 가까운 미래에 UAV를 활용한 서비스 시장 규모는 더욱 확대될 것으로 전망된다[36]. 이렇게 시장이 확대되는 상황에서 UAV에 대한 사이버 보안 이슈가 대두되고 있다. 특히, 군수분야 목적의 UAV는 국가 근간에 영향을 줄 수 있는 매우 민감한 정보를 다루기 때문에 보안의 중요성은 더욱 강조되고 있다.

본 논문에서는 미래전에 대비한 군수분야 UAV 환경에서의 보안을 지원하기 위해 기존 드론의 분류체계를 재정비하고, 군수분야 UAV의 보안 요구사항 즉 상호인증, 강력한 키교환, 기밀성, 무결성, 부인봉쇄, PFS, PBS, 서비스 거부 공격 방어, 그리고 중간자 공격 방어를 제시하였다. 특히 제안된 프로토콜은 군이라는 특수한 환경에서 작전을 수행할 때 이에 대한 책임소재를 명확히 할 필요가 있기 때문에 부인봉쇄가 필수적이다. 또한, 본 논문의 핵심으로 D-GCS 및 D-MD 두 가지의 시나리오를 위한 보안 프로토콜을 설계하였다. 초기 D-GCS 시나리오에서 강력한 보안성 지원을 통해 타 논문에서는 언급하지 않은 특수한 시나리오인 D-MD를 제시함으로써 각 UAV 간 안전한 통신을 경량화 하였다.

제안 프로토콜을 정형화 검증 도구인 Scyther와 양상 논리인 BAN 논리를 활용하여 검증하였으며, 이를 통해 제안 프로토콜이 상기 보안 요구사항을 만족함과 동시에 주요 공격에 안전함을 입증하였다. 또한, 연산 오버헤드 비교를 통해 초기 인증에서 강력한 보안을 제공한 후, 이후 연결될 D-MD 통신에서 상대적으로 가벼운 연산만으로 인증을 제공할 수 있음을 보였다.

본 논문의 향후 연구로서 제안한 프로토콜의 효율성과 실용성 검증을 위해 실장비 기반의 프로토콜 구현 및 실증 테스트를 할 예정이고, 군수분야의 다양한 세부 UAV 활용 시나리오 연구 및 시나리오별 프로토콜 확장을 연구할 계획이다.

참 고 문 헌

- [1] 신동훈, “4차 산업시대 핵심 왜 드론인가?”, cctvnews, <http://www.cctvnews.co.kr/news/articleView.html?idxno=86305>, August 2018.
- [2] 김기휘, “4차 산업혁명 시대 신성장동력으로 떠오른 드론 산업”, 고양신문, <http://www.mygoyang.com/news/articleView.html?idxno=45404>, 2018.
- [3] 김익현, “현란한 드론쇼…첨단 IT올림픽 막을랐다”, ZDNet Korea, <http://www.zdnet.co.kr/view/?no=20180209231102>. January 2018.
- [4] 이성엽, “드론 기술현황 및 기술경쟁력 분석”, 산은조사월보, pp. 88 December 2016.
- [5] 이상원, 김순석, 김양현, “경찰활동에서의 드론(무인항공기) 활용방안과 그 법적 문제 연구”, 서울 : 치안정책연구소, pp. 233, 2016.
- [6] 김대진, “드론(Drone) 부상이 산업에 미치는 영향과 시사점”, KDB산업은행, 2015.
- [7] 산업통상자원부, “드론로봇분야 민군협력방안,” 제3차 드론로봇 민군협력 협의회, 2018.
- [8] 김홍우, “SF 영화같은 벌떼 공격 현실로...‘드론봇’이 최전선서 싸운다”, 서울경제, <https://www.sedaily.com/NewsView/10ORZCRDRH>, December 2017.
- [9] 신동연, “‘게임 체인저’ 군사용 드론, 어디까지 개발됐나”, 중앙일보, <https://news.joins.com/article/23031203>, October 2018
- [10] 유성민, “드론은 해킹에 얼마나 취약할까?”, The Science Times,

- <https://www.sciencetimes.co.kr/?news=드론은-해킹에-얼마나-취약할까>,
December 2016.
- [11] 박현정, “사물인터넷 해킹 당하면?...내 차 조종당하고, 날던 드론 떨어지고”,
한겨레, <http://www.hani.co.kr/arti/economy/it/748425.html#csidx90919a333de804b84a01f4a2dec6637>, June 2016.
- [12] Galois, Inc, “Galois Embedded Crypto: Light Weight Cryptography,”
<https://github.com/GaloisInc/gec/blob/master/README.md>, April 2015.
- [13] Lorenz Meier, “Micro Air Vehicle Communication Protocol(MAVLink)”,
<https://mavlink.io>, 2009.
- [14] Lorenz Meier, “Secure MAVLink(sMAVLink)”, <https://diydrones.com/profiles/blogs/smaavlink-secure-mavlink-request-for-comments>, August 2013.
- [15] NIST, “Special Publication 800-56A, Recommendation for Pair-Wise Key
Establishment Schemes Using Discrete Logarithm Cryptography”,
<https://csrc.nist.gov/publications/detail/sp/800-56a/revised/archive/2007-03-14>,
March 2006.
- [16] Brecht Wyseur, “White-Box Cryptography”, PhD thesis, Katholieke
Universiteit Leuven, B. Preneel (promotor), pp. 169-201, March 2009.
- [17] S.H Seo, J. Won, E. Bertino, Y. Kang, and D. Choi, “A security

- framework for a drone delivery service,” in Proceedings of the 2Nd Workshop on Micro Aerial Vehicle Networks, Systems, and Applications for Civilian Use, ser. DroNet '16. ACM, pp. 29–34, 2016.
- [18] O. Blazy, P.-F. Bonnefoi, E. Conchon, D. Sauveron, R. N. Akram, K. Markantonakis, K. Mayes and S. Chaumette, "An efficient protocol for uas security", 2017 Integrated Communications Navigation and Surveillance (ICNS), 2017.
- [19] M. Wazid, A. K. Das, N. Kumar, A. V. Vasilakos, and J. J. P. C. Rodrigues, "Design and analysis of secure lightweight remote user authentication and key agreement scheme in Internet of drones deployment," IEEE Internet of Things, December 2018.
- [20] 김경수, 김지훈, “미래전을 주도할 군집 드론(Drone Swarm) 개발동향 및 발전 추세”, 국방과 기술 제479호, pp. 98–109, 2019.
- [21] 김황남, “공중 보안을 위한 안티 드론 기술”. 2018 KRNet, June 2018.
- [22] 강유성, 김주한, 김건우, “드론 기반 서비스를 위한 보안 요구사항”, 한국정보통신기술협회, vol.177, pp. 74–79, May 2018.
- [23] Cremers. C, “Scyther-Semantics and verification of security protocols”. PhD dissertation; Eindhoven University of technology. 2006.

- [24] M. Burrows, M. Abadi, and R. Needham, "A logic of authentication," ACM Trans. Comput. Syst., vol. 8, no. 1, pp. 18 - 36, February 1990.
- [25] Pascal Brisset, Antoine Drouin, Michel Gorraz, PierreSelim Huard, Jeremy Tyler, "The paparazzi solution", MAV 2006, Sandestin, Florida, 2006.
- [26] J. Salowey, A. Choudhury, D. McGrew, " AES Galois Counter Mode (GCM) Cipher Suites for TLS", IETF RFC 5288, <https://tools.ietf.org/html/rfc5288> [Online; Accessed on September 30, 2019], August 2008.
- [27] O'Higgins. B, Diffie. W, Strawczynski. L, do Hoog. R, "Encryption and ISDN - A Natural Fit", 1987 International Switching Symposium (ISS87), 1987.
- [28] Daniel J. Bernstein, "Curve25519: new Diffie-Hellman speed records", International Workshop on Public Key Cryptography, 2006.
- [29] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, Bo-Yin Yang. "ed25519: High-speed high-security signatures," Journal of Cryptographic Engineering 2 (2012), pp. 77-89, 2011.
- [30] Lyudmil L, "MD5, SHA-1, SHA-256 and SHA-512 speed performance", Automation Rhapsody, May 2018.

- [31] Liang X, Witit T, “Advanced Encryption Standard (AES) in Counter Mode”, ECE 575 Course Project, 2004.
- [32] D Johnson, A Menezes, S Vanstone, “The Elliptic Curve Digital Signature Algorithm (ECDSA),” International Journal of Information Security, August 2001.
- [33] Dolev. D, Yao A. C, "On the security of public key protocols," IEEE Transactions on Information Theory, pp. 298-208 , 1983
- [34] Cremers. C, “The Scyther Tool: Verication, Falsication, and Analysis of Security Protocols?,” Department of Computer Science, ETH Zurich, Switzerland Proceedings of the 20th International Conference on Computer Aided Verification (CAV 2008), Princeton, USA, 2008.
- [35] Cremers. C, Mauw. S, “Security properties. In: Operational Semantics and Verification of Security Protocols”, Springer, pp. 37 - 65, 2012.
- [36] Business Insider, “IHS Janes Intelligence Review,” GSV Asset Management, 2016.

부 록

본 부록에서는 본 논문에서 제안한 미래전에 대비한 군수분야 UAV 환경에서의 보안 프로토콜을 검증하기 위하여 활용한 Scyther의 소스코드를 포함한다.

```
var D, GCS, MD : Agent;
```

```
const idMission, idD, idMD, idGCS, P : Nonce;
```

```
hashfunction HM, h, g;
```

```
const pkD, pkGCS : Function;
```

```
secret skD, skGCS : Function;
```

```
inversekeys (pkD, skD);
```

```
inversekeys (pkGCS, skGCS);
```

```
macro kMD = k(MD, GCS);
```

```
protocol @oracle(DH) {
```

```
..... role DH{
```

```
.....
```

var x, y: Nonce;

.....
recv_!DH1(DH, DH, h(g(y),x));

.....
send_!DH2(DH, DH, h(g(x),y));

..... }

}

protocol @executability(O) {

.....role O {

.....
secret ekGCS, akGCS, ekD, ekD2, akD, akD2, ekMD, akMD, SK : SessionKey;

.....
fresh certGCS, certD, CMD, ts1, ts2, ts3, ts4, ts5, ts6, ts7, dGCS, dD, dD2,

dMD, qGCS, qD, qD2, qMD : Nonce;

.....

var S1, S2, S3, S4, stD, encD : Nonce;

.....

recv_!O1(O, O, idMission, certD, g(dD), ts1, S1, HM(P, idMission, certD, g(dD),
ts1, S1));

.....

send_!O2(O, O, idMission, certD, qD, ts1, S1, HM(P, idMission, certD, qD, ts1,
S1));

.....

recv_!O3(O, O, idMission, certGCS, g(dGCS), ts2, S2, {CMD}ekGCS
,HM(akGCS, idMission, certGCS, g(dGCS), ts2, S2, {CMD}ekGCS));

.....

send_!O4(O, O, idMission, certGCS,qGCS, ts2, S2, {CMD}ekD ,HM(akD,
idMission, certGCS, qGCS, ts2, S2, {CMD}ekD));

.....

.....

recv_!O5(O, O, idMission, idD, idGCS, ts3, S3, HM(akD, idMission, idD, idGCS,
ts3, S3));

.....

send_!O6(O, O, idMission, idD, idGCS, ts3, S3, HM(akGCS, idMission, idD,
idGCS, ts3, S3));

.....

recv_!O7(O, O, idMission, idGCS, idD, ts4, S4, encD, stD, HM(akGCS,
idMission, idGCS, idD, ts4, encD, stD, S4));

.....

send_!O8(O, O, idMission, idGCS, idD, ts4, S4, encD, stD, HM(akD, idMission,
idGCS, idD, ts4, encD, stD, S4));

.....

recv_!O9(O, O, idMission, stD, idD, g(dD2), ts5, HM(idMission, stD, idD,
g(dD2), ts5));

.....

send_!O10(O, O, idMission, stD, idD, qD2, ts5, HM(idMission, stD, idD, qD2,
ts5));

.....

recv_!O11(O, O, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD,
ts6), HM(SK, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD,
ts6)));

.....

send_!O12(O, O, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD,
g(dMD), ts6), HM(SK, idMission, idMD, g(dMD), ts6, HM(akMD, idMission,
idMD, g(dMD), ts6)));

.....

recv_!O13(O, O, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD,
ts7));

.....

send_!O14(O, O, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD,
ts7));

```

.....}

}

```

```

protocol MUSP(D, GCS, MD) {

```

```

..... role D{

```

```

.....

```

```

fresh certD, dD, dD2 : Nonce;

```

```

.....

```

```

fresh ts1, ts3, ts5, ts7 : Nonce;

```

```

.....

```

```

var qGCS, CMD, certGCS, LT, qMD, ts2, ts4, ts6 : Nonce;

```

```

.....

```

```

var stD : Ticket;

```

```

.....

```

```

var SK : SessionKey;

```

.....

macro S1D = {idMission, certD, g(dD), ts1}skD;

.....

macro S2D = {idMission, certGCS, qGCS, ts2}skGCS;

.....

macro S3D = {idMission, idD, idGCS, ts3}skD;

.....

macro mskD = g(qGCS, dD);

.....

macro mskD2 = g(qMD, dD2);

.....

macro ekD = HM(mskD, ts1);

.....

macro ekD2 = HM(mskD2, ts5);

```

.....

macro akD = HM(mskD, ts1);

.....

macro akD2 = HM(mskD2, ts5);

.....

macro encD = {idD, idMD, idGCS, SK, LT, ts4}ekD;

.....

macro S4D = {idMission, idGCS, idD, ts4, encD, stD}skGCS;

.....

send_!1(D, GCS, idMission, certD, g(dD), ts1, S1D, HM(P, idMission, certD,
g(dD), ts1, S1D));

.....

recv_!2(GCS, D, idMission, certGCS,qGCS, ts2, S2D, {CMD}ekD ,HM(akD,
idMission, certGCS, qGCS, ts2, S2D, {CMD}ekD));

.....

claim(D, Running, GCS, HM(akD, ts3, S3D));

```

.....

send_!3(D, GCS, idMission, idD, idGCS, ts3, S3D, HM(akD, idMission, idD,
idGCS, ts3, S3D));

.....

recv_!4(GCS, D, idMission, idGCS, idD, ts4, S4D, encD, stD, HM(akD,
idMission, idGCS, idD, ts4, encD, stD, S4D));

.....

claim(D, Secret, CMD);

.....

claim(D, Secret, mskD);

.....

claim(D, Secret, akD);

.....

claim(D, Secret, ekD);

.....

claim(D, Commit, GCS, HM(akD, ts3, S3D));

.....

.....
send_!5(D, MD, idMission, stD, idD, g(dD2), ts5, HM(idMission, stD, idD,
g(dD2), ts5));

.....
recv_!6(MD, D, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD,
ts6), HM(SK, idMission, idMD, qMD, ts6, HM(akD2, idMission, idMD, qMD,
ts6)));

.....
claim(D, Running, MD, HM(akD2, ts7));

.....
send_!7(D, MD, idMission, idD, idMD, ts7, HM(akD2, idMission, idD, idMD,
ts7));

.....
claim(D, Alive);

.....
claim(D, Nisynch);

.....

claim(D, Niagree);

.....

claim(D, Weakagree);

.....

claim(D, Secret, mskD2);

.....

claim(D, Secret, akD2);

.....

claim(D, Secret, ekD2);

.....

claim(D, Commit, MD, HM(akD2, ts7));

..... }

..... role GCS{

.....

fresh certGCS, dGCS, ts2, ts4, LT: Nonce;

.....

var certD, qD, ts1, ts3 : Nonce;

.....

secret CMD : Nonce;

.....

fresh SK : SessionKey;

.....

macro S1GCS = {idMission, certD, qD, ts1}skD;

.....

macro S2GCS = {idMission, certGCS, g(dGCS), ts2}skGCS;

.....

macro S3GCS = {idMission, idD, idGCS, ts3}skD;

.....

macro mskGCS = g(qD, dGCS);

.....

macro ekGCS = HM(mskGCS, ts1);

.....

macro akGCS = HM(mskGCS, ts1);

.....

.....

macro stD = {idMission, idD, idMD, idGCS, SK, LT}kMD;

.....

macro encD = {idD, idMD, idGCS, SK, LT, ts4}ekGCS;

.....

macro S4GCS = {idMission, idGCS, idD, ts4, encD, stD}skGCS;

.....

recv_!1(D, GCS, idMission, certD, qD, ts1, S1GCS, HM(P, idMission, certD, qD,
ts1, S1GCS));

.....

send_!2(GCS, D, idMission, certGCS, g(dGCS), ts2, S2GCS, {CMD}ekGCS
,HM(akGCS, idMission, certGCS, g(dGCS), ts2, S2GCS, {CMD}ekGCS));

.....

recv_!3(D, GCS, idMission, idD, idGCS, ts3, S3GCS, HM(akGCS, idMission,
idD, idGCS, ts3, S3GCS));

.....
claim(GCS, Running, D, HM(akGCS, ts4, S4GCS));

.....
send_!4(GCS, D, idMission, idGCS, idD, ts4, S4GCS, encD, stD, HM(akGCS,
idMission, idGCS, idD, ts4, encD, stD, S4GCS));

.....
claim(GCS, Alive);

.....
claim(GCS, Nisynch);

.....
claim(GCS, Niagree);

.....
claim(GCS, Weakagree);

.....
claim(GCS, Secret, CMD);

.....
claim(GCS, Secret, mskGCS);

```

.....

claim(GCS, Secret, akGCS);

.....

claim(GCS, Secret, ekGCS);

.....

claim(GCS, Commit, D, HM(akGCS, ts4, S4GCS));

..... }

..... role MD{

.....

fresh dMD, ts6 : Nonce;

.....

var qD2, LT, qD2, ts5, ts7 : Nonce;

.....

var SK : SessionKey;

.....

```

```
macro stD = {idMission, idD, idMD, idGCS, SK, LT}kMD;
```

```
.....
```

```
macro mskMD = g(qD2, dMD);
```

```
.....
```

```
macro akMD = HM(mskMD, ts5);
```

```
.....
```

```
macro ekMD = HM(mskMD, ts5);
```

```
.....
```

```
recv_!5(D, MD, idMission, stD, idD, qD2, ts5, HM(idMission, stD, idD, qD2,  
ts5));
```

```
.....
```

```
claim(MD, Running, D, HM(akMD, ts6));
```

```
.....
```

```
send_!6(MD, D, idMission, idMD, g(dMD), ts6, HM(akMD, idMission, idMD,  
g(dMD), ts6), HM(SK, idMission, idMD, g(dMD), ts6, HM(akMD, idMission,  
idMD, g(dMD), ts6)));
```

.....
recv_!7(D, MD, idMission, idD, idMD, ts7, HM(akMD, idMission, idD, idMD,
ts7));

.....
claim(MD, Alive);

.....
claim(MD, Nisynch);

.....
claim(MD, Niagree);

.....
claim(MD, Weakagree);

.....
claim(MD, Secret, mskMD);

.....
claim(MD, Secret, akMD);

.....
claim(MD, Secret, ekMD);

```
.....  
  
claim(MD, Commit, D, HM(akMD, ts6));  
  
..... }  
  
}
```

ABSTRACT

ESPMU: an Efficient Security Protocol in the Military UAV Environment for the Future War

Yongho Ko

Department of Information Security Engineering
Graduate School of Soonchunhyang University
Asan, Korea

Supervised by Ilsun You

The future of war is transforming into an informational advantage that can be won only if fast commands and preemptive strikes are prepared under the NCOE(Network Centric Operational Environment). In order to realize rapid commands, it is necessary to find ways to collect information under various circumstances such as land-based, air and sea. One such way of achieving swift commands is demonstrated through UAVs(Unmanned Aerial Vehicle) which has become a representative game changers in this area. The UAV refers to a flight system that is capable of flying without an inboard pilot (“unmanned”) and often recognize the environment autonomously. UAVs find numerous applications in various fields such as military, aerospace, reconnaissance, agriculture, and many more. Unfortunately, as they became widespread and their demand grows, they are becoming more and more vulnerable to several security attacks including, but not limited to, jamming, information leakage, and spoofing attacks. In order to cope with such attacks and security threats, a proper design of robust security protocols is essential. Although a number of researches have been carried out, there are still research gaps especially in relation to D2D(Drone-to-Drone) secure communication, which is regarded as a core element of the UAV communication in military applications. In this paper, we propose a security protocol for a military UAV. Accordingly, we first introduce the concept of military UAV, define security requirements, and present a threat model that is applicable for the military UAV. Based on the defined security requirements, we propose a security protocol for D2D and Drone-to-Ground Control Station communications. Finally, the security of the proposed protocol is verified through Scyther and BAN logic, and the performance of the protocol is shown by comparing the computational overhead with other related protocols. The proposed protocol is expected to effectively support UAV conduct of operations and operational control through strong authentication, key exchange, digital signature-based non-repudiation and accountability tracking, and secure D2D security with perfect forward secrecy.

Keyword: Military UAV, Authentication, Key Management, Security,

Formal Verification, Scyther, BAN Logic